

VINUNIVERSITY

Multi-Agent Systems 2026

AICB-P2T3 · Ngày 20 · Chương 4 — Từ hệ đa agent đến harness, tác vụ dài hạn và agent tự tiến hoá

Giảng viên

VinUniversity · Phase 2 · Track 3 · Tuần 4

HÃY SUY NGHĨ...

*"**16 agent** chạy song song, chi phí **20.000 USD**, tạo ra một trình biên dịch C biên dịch được nhân Linux.*

*Một nghiên cứu khác: với tác vụ lập kế hoạch tuần tự, hệ đa agent **kém hơn một agent tới 70%**.*

Khi nào nhiều agent làm kết quả tốt hơn, và khi nào làm kết quả tệ hơn?"

Giữ câu hỏi này trong đầu khi học bài hôm nay

Mười chương, mỗi chương trả lời một câu hỏi

Ch.	Chủ đề	Câu hỏi của chương	phút
1	Một agent hay nhiều agent?	Khi nào tách thành nhiều agent cho kết quả tốt hơn?	25
2	Mẫu thiết kế phối hợp	Chọn mẫu phối hợp nào cho một bài toán cụ thể?	20
3	Kiểu lỗi của hệ đa agent	Hệ đa agent (multi-agent) hỏng ở đâu, và sửa từ đâu?	15
4	Harness	Phần mềm bao quanh mô hình quyết định điều gì?	25
5	**DeepSeek Harness**	**Thiết kế thế nào, hoạt động ra sao, vì sao được chú ý?**	30
6	Thiết kế ngữ cảnh giữa các agent	Cái gì được chuyển, bị cắt, hay còn lại sau khi đặt lại ngữ cảnh?	15
7	Agent chạy dài hạn	Điều gì giúp agent làm việc hàng giờ mà không lạc mục tiêu?	25
8	Agent tự tiến hoá	Thực chất thay đổi cái gì, và khi nào đáng giữ?	25
9	Đánh giá, quan sát và bảo mật	Làm sao biết hệ đa agent đúng, và giữ nó an toàn?	15
10	Thực hành và tổng kết	Áp dụng vào một bài toán cụ thể và đo kết quả thế nào?	20

Cách đọc bộ slide

Mỗi chương (trừ chương 10) mở bằng một slide **ý chính**. Tiêu đề các slide sau là khẳng định cần chứng minh. Nhãn ●●○ Nâng cao đánh dấu phần nâng cao, có thể bỏ qua khi thiếu thời gian.

Vị trí trong Track 3

Ngày 16–19 xây *một* agent tốt. Hôm nay ghép nhiều vòng lặp và xác định khi nào không nên ghép. LangGraph: Ngày 23; MCP và A2A: Ngày 26. Tổng 220 phút; quiz và lab 20 học riêng.

Chương 1 — Một agent hay nhiều agent?

Câu hỏi của chương: **Khi nào tách thành nhiều agent cho kết quả tốt hơn một agent?**

Hệ đa agent (multi-agent) chỉ đáng dùng khi việc chia được thành các phần độc lập hoặc khi một cửa sổ ngữ cảnh (context window) không đủ chứa; ngoài hai trường hợp đó, nó làm tăng chi phí và lỗi mà không tăng chất lượng.

1. **Tách để chia ngữ cảnh.** Agent (hệ thống AI tự chọn hành động và công cụ để hoàn thành mục tiêu) đọc nhiều làm đầy ngữ cảnh (context).

2. **Cấu trúc bài toán quyết định.** Trong 260 cấu hình, hệ đa agent tăng 80,8% ở bài phân rã được, giảm 70,0% ở bài tuần tự, so với hệ một agent (single-agent).

3. **Chất lượng phụ thuộc lượng token.** Hệ đa agent dùng khoảng 15 lần token (đơn vị văn bản nhỏ nhất mà mô hình xử lý) của một lượt hội thoại.

Sau chương này: áp dụng năm câu hỏi để quyết định giữ một agent hay tách.

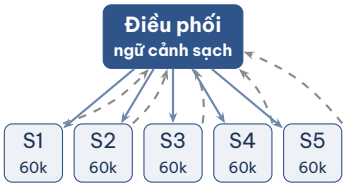
Tách agent khi một cửa sổ ngữ cảnh không đủ chứa

Mỗi agent là một mô hình (model) gọi công cụ (tool) trong vòng lặp agent (agent loop); harness (phần mềm bao quanh mô hình: quyết định mô hình thấy gì, làm được gì, khi nào dừng) cấp cho mỗi agent một cửa sổ ngữ cảnh riêng.

Một agent đọc tất cả



Một agent điều phối + 5 subagent



mỗi subagent đọc ~60k, **trả về 1,5k**

Phép tính

30 nguồn × 10k = **300k** token. Chia cho 5 subagent (agent phụ do agent chính tạo ra để làm một phần việc): mỗi subagent đọc 60k, trả 1,5k; agent điều phối (orchestrator) nhận **7,5k** token. Tổng token *tăng*, không giảm.

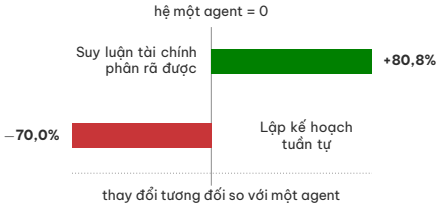
Điều kiện tách

Chia vai khi ngữ cảnh chưa đầy chỉ thêm chi phí và điểm sai; đây là điều kiện thứ hai của luận điểm.

Nguồn: Anthropic, *How we built our multi-agent research system* (13/06/2025) — subagent đóng vai bộ lọc nén thông tin.

Bài phân rã được: +80,8%; bài tuần tự: −70,0%

Kim et al. (Google Research, v3): 5 kiến trúc × 3 họ mô hình × 6 bộ đánh giá chuẩn (benchmark).



Đọc kết quả

Cấu trúc bài toán quyết định, không phải số agent.

Phần độc lập thì lợi lớn; bước sau phụ thuộc bước trước thì chuyển giao (handoff) làm rơi thông tin.

87%

bộ dự đoán chọn đúng kiến trúc trên cấu hình chưa thấy

> 45%

hệ một agent đã vượt mức này thì thêm agent ít lợi

17,2× so với 4,4×

khuếch đại lỗi: agent độc lập so với có agent điều phối kiểm

Giới hạn bằng chứng

Một nghiên cứu: coi là giả thuyết cần đo. Kết quả ủng hộ trường hợp thứ nhất của luận điểm.

Nguồn: arXiv:2512.08296 v3 — tóm tắt (260 cấu hình, +80,8%/−70,0%, 87%) và toàn văn (45%, 17,2× so với 4,4×).

Chất lượng tăng 90,2%, token gấp khoảng 15 lần

+90,2%

so với một agent Opus 4, theo đánh giá (evaluation) **nội bộ** của Anthropic

≈15×

token so với một lượt hội thoại (hệ một agent: ≈4×)

80%

phương sai kết quả BrowseComp được giải thích bằng **lượng token**

Phép tính cho một sản phẩm

Một câu hội thoại ≈ **2.000** token; hệ một agent ≈ 8.000; hệ đa agent ≈ **30.000**.

10.000 câu mỗi ngày: 20M → 300M token. Nếu chỉ 5% số câu chuyển sang hệ đa agent: $0{,}95 \times 2\text{k} + 0{,}05 \times 30\text{k} =$ **3,4k** token mỗi câu, thay vì 30k.

Phù hợp khi — giá trị cao, đọc rộng, song song được, vượt một cửa sổ ngữ cảnh, nhiều công cụ.

Lưu ý: 90,2% là đánh giá nội bộ, chưa tái lập được.

Hệ quả cho luận điểm

Phần lớn mức tăng chất lượng đến từ lượng token dùng thêm, không đến từ việc chia nhiều agent.

Nguồn: Anthropic, *How we built our multi-agent research system* (13/06/2025).

Năm câu hỏi quyết định có thêm agent hay không

	Câu hỏi	Nếu trả lời "không"	Đo bằng
1	Việc tách được thành các phần **độc lập**?	giữ một agent	đồ thị phụ thuộc
2	Mỗi agent nhận **đủ ngữ cảnh** để làm đúng?	viết brief (bản mô tả việc giao cho subagent) trước	đọc brief như người ngoài
3	Kết quả **ghép** lại được, không xung đột ngầm?	chỉ cho một agent ghi	tỉ lệ xung đột khi gộp
4	Có bước **xác minh** (verification) trước khi tổng hợp?	thêm bước xác minh trước	tỉ lệ lỗi lọt qua
5	Lợi ích có đáng $\approx$**4**$\times$ token của hệ một agent?	chỉ chuyển phần việc khó	chi phí mỗi câu trả lời *được chấp nhận*

Bước 0 — chạy hệ một agent tốt nhất với cùng công cụ và cùng ngân sách token; nếu đã trên $\sim$45%, lợi ích biên của việc thêm agent nhỏ.

Kết luận chương

Tách khi câu 1 là "có" hoặc một cửa sổ ngữ cảnh không đủ chứa; khi đã tách, song song hoá việc đọc, tuần tự hoá việc ghi.

Nguồn: cognition.com/blog/dont-build-multi-agents (2025); cognition.com/blog/multi-agents-working (22/04/2026); Anthropic (13/06/2025).

Câu hỏi của chương: **Chọn mẫu phối hợp nào cho một bài toán cụ thể?**

Chọn mẫu phối hợp theo hình dạng phụ thuộc giữa các tác vụ: bắt đầu từ luồng công việc (workflow) cố định, chỉ nâng lên mẫu điều phối–thực thi (orchestrator–worker) khi các nhánh thật sự độc lập, và đo mọi mẫu phức tạp hơn so với đối chứng đơn giản trên cùng số câu trả lời.

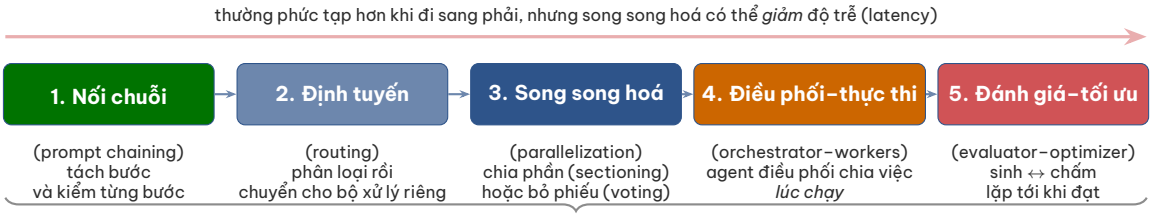
1. **Luồng công việc trước.** Năm mẫu thiết kế (pattern) của Anthropic đều do mã điều khiển; mô hình chỉ tự điều khiển khi đường đi không đoán trước được.

2. **Hình dạng phụ thuộc chọn cấu trúc.** Sáu cấu trúc kết nối (topology) có điểm hỏng khác nhau; agent điều phối chỉ giao lại phần thiếu.

3. **Mẫu phức tạp phải được đo.** Cùng số câu trả lời, tự nhất quán (self-consistency) đạt 88,2%; tranh luận giữa các agent (debate) đạt 83,0%.

Sau chương này: chọn mẫu theo đồ thị phụ thuộc và viết khung mã điều phối có hạn giờ.

Luồng công việc cố định trước, agent tự chủ sau



Hai trục độc lập

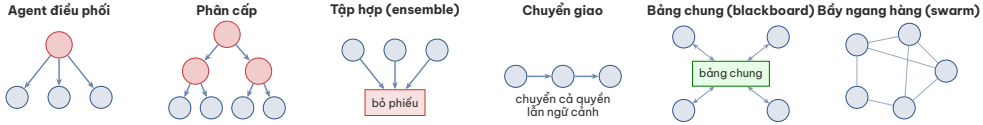
Ai điều khiển vòng lặp: mã (luồng công việc) hay mô hình (agent tự chủ). **Số vòng lặp agent phối hợp**: một hay nhiều.

Quy tắc

Dùng mẫu đơn giản nhất đáp ứng được việc; chỉ nâng lên mẫu tiếp theo khi đo được lợi ích.

Nguồn: Anthropic, *Building effective agents* (19/12/2024).

Sáu cấu trúc kết nối, mỗi cấu trúc một điểm hỏng



Cấu trúc	Hợp khi	Rủi ro chính
Agent điều phối (supervisor)	việc tách được, có bước gộp rõ	agent điều phối là nút cổ chai; chia việc sai thì mọi nhánh sai
Phân cấp	bài rất lớn, miền con ổn định	thêm tầng là thêm một lần chuyển giao
Tập hợp	cần nhiều phương án độc lập	lỗi tương quan; dùng mô hình khác họ làm bộ phản biện
Chuyển giao	đưa khách hàng sang chuyên viên	mất ngữ cảnh ở chỗ giao
Bảng chung	cần một bản ghi chung lâu dài	một thông tin sai làm nhiễm mọi agent
Bầy ngang hàng	khám phá mở	không ai chịu trách nhiệm; Cognition: gây xao nhãng

Quy tắc chọn

Chọn cấu trúc theo đồ thị phụ thuộc của việc, rồi kiểm tra điểm hỏng riêng của cấu trúc đó.

Nguồn: Tổng hợp từ arXiv:2512.08296; Claude Managed Agents docs; OpenAI multi-agent docs (04/10/2026); Cognition (04/2026).

Điều phối–thực thi: chỉ giao lại phần còn thiếu



Brief gồm

Agent thực thi (worker) nhận: **mục tiêu** (một câu) · **định dạng** kết quả trả về · **công cụ** và nguồn được dùng · **ranh giới**: phần *không* làm, để khỏi trùng việc nhánh khác.

Công sức co giãn theo độ khó

Hỏi một sự kiện: 1 agent, 3–10 lời gọi công cụ (tool call). So sánh: 2–4 subagent. Nghiên cứu phức tạp: hơn 10 subagent, mỗi nhánh một việc rõ ràng.

Lưu ý: Anthropic ghi nhận: giao việc "tìm hiểu thiếu chip" quá mơ hồ khiến các subagent làm trùng nhau, có nhánh nghiên cứu nhầm giai đoạn lịch sử.

Nguồn: Anthropic, *How we built our multi-agent research system* (13/06/2025) — mục "Prompt engineering and evaluations".

Khung mã điều phối có hạn giờ và kết quả dở dang


```
sem = asyncio.Semaphore(4)                          # ⩽ rate limit
async def orchestrate(question, max_rounds=3):
  plan = await lead.plan(question); memory.save("plan", plan)
  findings = []
  for rnd in range(max_rounds):                              # hard stop
    results = await asyncio.gather(
      *[worker(b) for b in plan.briefs], return_exceptions=True)
    findings += [r for r in results if ok(r)]
    log_failures(r for r in results if not ok(r))    # never silent
    gaps = await lead.verify(question, findings)
    if not gaps: return await lead.synthesize(question, findings)
    plan = await lead.replan(gaps)                  # only the gaps
  return partial(findings, unresolved=gaps)  # say it is incomplete

async def worker(brief):
  async with sem:
    ctx = [SYSTEM, brief.render()]                  # fresh context
    return await asyncio.wait_for(                          # deadline
      agent_loop(ctx, brief.tools, max_tool_calls=15), 120)
```

Sự cố → cơ chế

Agent treo: `wait_for` đặt hạn giờ.
Vượt giới hạn tần suất (rate limit): `Semaphore`.
Lỗi tạm thời: thử lại (retry) có giới hạn; lỗi lặp lại thì đổi agent.
Sai lược đồ dữ liệu (schema): trả thông báo lỗi nêu cách sửa, tính là thất bại.
Hết vòng mà còn thiếu: trả `partial` kèm `gaps`.

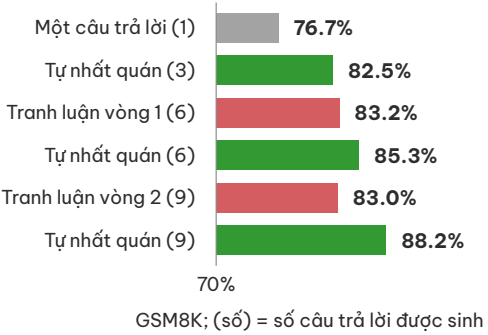
Chốt

Kết quả dở dang không được trình bày như đã xong.

Nguồn: Python `asyncio` docs (`gather`, `wait_for`); Anthropic (13/06/2025) — mục về vận hành.

Cùng số câu trả lời, tự nhất quán hơn tranh luận

Con số hay được trích "tranh luận giảm ảo giác (hallucination) 15–25%" không có nguồn. Dữ liệu thật:



Đọc biểu đồ

Tranh luận: 3 agent sinh 3 câu đầu, mỗi vòng thêm 3 câu, nên vòng 1 có **6** và vòng 2 có **9** câu trả lời. Ở cùng số câu, **tự nhất quán thắng**: 85,3 so với 83,2 và **88,2 so với 83,0**.

Chưa phải cùng ngân sách

Câu lệnh đầu vào (prompt) của tranh luận chứa câu trả lời của agent khác nên tốn token hơn. Smit et al. (ICML 2024): tranh luận không vượt tập hợp ổn định.

Quy tắc đối chứng

Chỉ mở tranh luận khi các mẫu độc lập *bất đồng*; tranh luận chỉ "thắng" khi so với một câu trả lời duy nhất.

Nguồn: Huang et al., *LLMs Cannot Self-Correct Reasoning Yet* (ICLR 2024), Table 7; Du et al., arXiv:2305.14325; Smit et al., PMLR v235 (ICML 2024).

Định tuyến giảm 63% chi phí khi độ khó có hai đỉnh

VINUNIVERSITY

Phép tính

Mô hình nhỏ = 1, mô hình lớn = 10; 70% câu dễ, 30% câu khó: $0{,}7 \times 1 + 0{,}3 \times 10 = \mathbf{3{,}7}$ thay vì **10**, giảm **63%**. Bộ phân loại cũng sai: nhầm 10% câu khó làm 3% câu trả lời kém; cần đường **leo thang**.

Việc cần làm	Lớp công cụ	Ví dụ
Đồ thị trạng thái (state), điểm lưu (checkpoint), có con người phê duyệt (human-in-the-loop)	thư viện điều phối (orchestration)	LangGraph (Ngày 23)
Vòng lặp agent chạy ngay trong tiến trình của ứng dụng	SDK agent	Claude Agent SDK, OpenAI Agents SDK
Agent chạy hộ trên hạ tầng nhà cung cấp	dịch vụ agent được quản lý	Claude Managed Agents, OpenAI Agents API
Agent lập trình dùng ngay, mở rộng được	harness	Claude Code, Codex CLI, Pi, DeepSeek Harness
Agent khác hãng trao đổi với nhau	giao thức	A2A 1.0 (Ngày 26)

Chốt

Định tuyến là mẫu rẻ nhất có lợi thật nhưng phải đo tỉ lệ định tuyến sai.

Nguồn: MCP changelog 2026-07-28; A2A spec 1.0; tài liệu Claude Agent SDK, Managed Agents, OpenAI Agents API (truy cập 05/10/2026).

Câu hỏi của chương: **Hệ đa agent hỏng ở đâu, và sửa từ đâu?**

Ba nhóm lỗi của hệ đa agent (thiết kế hệ thống, phối hợp giữa các agent, xác minh) đều can thiệp được bằng cấu hình hệ thống hoặc cách giao việc, nên cần chẩn đoán theo phân loại lỗi (MAST) trước khi đổi mô hình.

1. **Phân loại lỗi.** MAST gom 14 kiểu lỗi (failure mode) thành ba nhóm: thiết kế hệ thống 44,2%, lệch giữa các agent 32,3%, xác minh (verification) và kết thúc 23,5%.

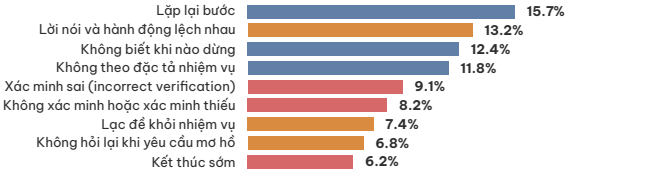
2. **Lỗi tích luỹ (error compounding).** Mỗi bước đúng 99% thì 100 bước còn 36,6%. Lỗi không xác minh bị khuếch đại 17,2 lần, còn 4,4 lần khi có kiểm trước khi gộp.

3. **Cách sửa.** Mỗi kiểu lỗi có một điểm sửa ngoài mô hình: cấu hình hệ thống hoặc cách giao việc. Thêm một bước xác minh mục tiêu cấp cao tăng ProgramDev 15,6%.

Sau chương này: đọc một vết thực thi (trace), xác định kiểu lỗi MAST và chọn điểm sửa trước khi đổi mô hình.

MAST: nhóm lỗi lớn nhất là thiết kế hệ thống

Cemri et al. (NeurIPS 2025): 1.642 vết thực thi từ 7 hệ đa agent mã nguồn mở; tỷ lệ thất bại 41%–86,7% tuỳ hệ.



9/14 kiểu lỗi phổ biến nhất (Hình 1); các kiểu còn lại $<3\%$ mỗi kiểu

Thiết kế hệ thống	**44,2%**
Lệch giữa các agent	**32,3%**
Xác minh và kết thúc	**23,5%**

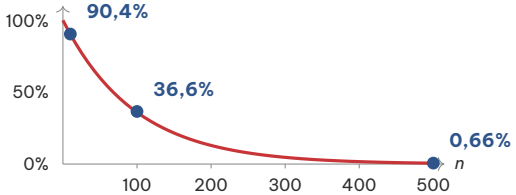
Lưu ý: "79% là lỗi đặc tả" hay được trích nhưng sai. Hình khác trong cùng bài cho 41,8 / 36,9 / 21,3 (tập vết thực thi khác); cần ghi rõ hình nào.

Phục vụ luận điểm của chương

Cả ba nhóm đều can thiệp được ở tầng thiết kế, phối hợp hoặc xác minh; chưa có so sánh trực tiếp với việc đổi mô hình. Vì vậy MAST là công cụ chẩn đoán đầu tiên.

Nguồn: Cemri et al., *Why Do Multi-Agent LLM Systems Fail?*, arXiv:2503.13657 v3, Figure 1 (1.642 vết thực thi), $\kappa = 0{,}88$.

Lỗi tích luỹ: 99% mỗi bước còn 37% sau 100 bước



Xác suất cả n bước đều đúng khi mỗi bước đúng 99%

Lỗi tích luỹ — xác suất cả chuỗi đúng bằng tích xác suất từng bước: $P = p^n$. Với $p = 0{,}99$, $n = 100$ thì $P = 36{,}6\%$.

17,2×
khuếch đại lỗi khi các agent độc lập

4,4×
khi agent điều phối kiểm trước khi gộp

Phục vụ luận điểm của chương

Hệ đa agent thêm hệ số khuếch đại vào chuỗi nhân; điểm xác minh cắt chuỗi đó. Phép tính giả định các bước độc lập; thực tế agent tự sửa được một phần, nhưng hướng của hiệu ứng vẫn đúng.

Nguồn: Kim et al., *Towards a Science of Scaling Agent Systems*, arXiv:2512.08296 v3 (khuếch đại lỗi ở mức vết thực thi); mô hình p^n là phép tính minh hoạ.

Mỗi kiểu lỗi có một điểm sửa ngoài mô hình

Kiểu lỗi (MAST)	Biểu hiện trong vết thực thi	Điểm sửa trong hệ thống hoặc cách giao việc
Lặp lại bước	gọi cùng một lệnh tìm kiếm 6 lần với truy vấn gần giống nhau	sổ tiến độ kèm phát hiện vòng lặp; giới hạn `max_tool_calls`
Không biết khi nào dừng	"kiểm thêm một lần nữa" cho đến hết ngân sách	tiêu chí xong viết ra được: danh sách việc, kiểm thử phải đạt
Không theo đặc tả	trả văn xuôi khi yêu cầu bảng JSON	kiểm tra đầu ra theo lược đồ dữ liệu; báo lỗi rõ để agent sửa
Lời nói lệch hành động	ghi "đã chạy kiểm thử" nhưng không có lệnh kiểm thử trong vết	xác minh bằng bằng chứng trong vết, không bằng lời tự báo cáo
Giữ lại thông tin	agent thực thi biết API đã đổi nhưng không báo agent điều phối	chỉ dẫn giao việc đòi mục "điều bất ngờ và rủi ro" trong kết quả trả về
Không xác minh, xác minh nông	agent xác minh chỉ kiểm mã biên dịch được	agent xác minh độc lập, đối chiếu với mục tiêu cấp cao

Một can thiệp, một con số

Thêm một bước xác minh mục tiêu cấp cao vào ChatDev, cùng mô hình, tăng 15,6% kết quả ProgramDev: mức tăng lớn nhất trong các can thiệp của bài, trên bộ tác vụ dễ, và tỷ lệ hoàn thành sau đó vẫn thấp.

Kiểm tra nhanh: chương 1–3

1. Hệ 3 agent đạt 78%, một agent đạt 70%. Cần thêm thông tin gì trước khi kết luận hệ 3 agent tốt hơn?
2. Vì sao "song song hoá việc đọc, tuần tự hoá việc ghi" giảm lỗi lệch giữa các agent?
3. Tranh luận giữa 3 agent, 2 vòng, nên so với mốc đối chứng nào để công bằng?
4. Vết thực thi cho thấy agent gọi `run_tests` rồi ghi `all tests pass`, nhưng đầu ra thật có 2 kiểm thử lỗi. Đó là kiểu lỗi MAST nào, và sửa ở đâu?
5. Một bài toán lập kế hoạch tuần tự có mốc một agent đạt 62%. Theo Kim et al., thêm agent có đáng thử không?

Gợi ý đáp án (giảng viên)

1: tổng token và chi phí mỗi bên, cùng bộ công cụ chưa, phương sai qua nhiều lần chạy. 2: chỉ một agent ra quyết định ghi, nên không có hai quyết định ngầm mâu thuẫn nhau. 3: tự nhất quán (self-consistency) với cùng số câu trả lời (2 vòng = 9) và đo cả token. 4: chủ yếu là xác minh sai; có thể nhận là lệch giữa lập luận (reasoning) và hành động; agent xác minh đọc mã thoát và đầu ra, không đọc lời tự báo cáo. 5: khả năng thấp, vì bài toán tuần tự và mốc đã trên 45%.

Câu hỏi của chương: **Phần mềm bao quanh mô hình quyết định điều gì?**

Một agent là mô hình cộng harness (phần mềm bao quanh mô hình); với cùng một mô hình, đổi harness làm kết quả đo được thay đổi, nên harness phải được thiết kế, đo và rà soát như mã nguồn.

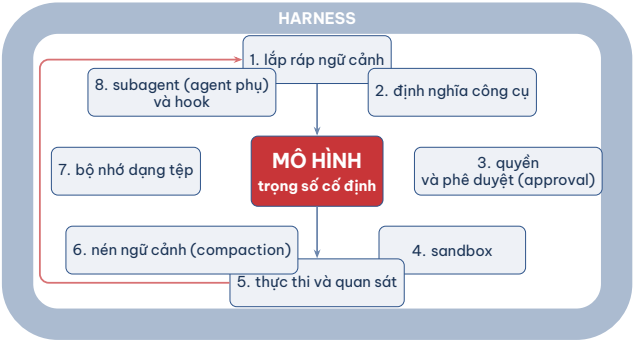
1. **Cùng mô hình, khác kết quả.** Chỉ đổi harness, số lời giải trọn vẹn tăng từ 43 lên 72 trên 169 tác vụ SWE-bench Verified.

2. **Chỉ dẫn không phải cưỡng chế.** Chỉ quyền (permission), sandbox (môi trường cách ly) và hook (mã chạy quanh lời gọi công cụ) mới ngăn được hành động; file hướng dẫn thì không.

3. **Công cụ là giao diện.** Giao diện agent–máy tính (ACI) tốt có đầu ra gọn và lỗi nêu cách sửa. Mười hai câu hỏi giúp chấm harness trước và sau khi sửa.

Sau chương này: đo harness bằng bộ tác vụ riêng và rà soát nó theo danh mục 12 mục.

Agent là mô hình cộng harness



Harness — phần mềm bao quanh mô hình, quyết định mô hình **thấy gì**, **làm được gì**, **chạy ở đâu** và **khi nào dừng**.

Vì sao coi là kỹ thuật phần mềm

Mô hình thường không đổi được; harness được viết, đo và sửa hằng ngày. Mô tả công cụ mơ hồ hay nhật ký 20.000 dòng đổ vào ngữ cảnh đều là lỗi của harness.

Phục vụ luận điểm của chương

Đơn vị được đánh giá là mô hình cộng harness. Một điểm của bộ đánh giá chuẩn mà không ghi harness là báo cáo thiếu.

Nguồn: Anthropic, *Effective context engineering for AI agents* (29/09/2025); OpenAI, *Harness engineering* (11/02/2026); Harness-Bench, arXiv:2605.27922.

Cùng một mô hình, đổi harness: 43 → 72 lời giải

VINUNIVERSITY

Thí nghiệm có kiểm soát (08/2026)

Cùng mô hình, 169 tác vụ SWE-bench Verified, ngữ cảnh 20.480 token, 480 giây. Chỉ đổi harness: rút gọn kết quả công cụ cũ khi ngữ cảnh đầy.



Kiểm thử chuyển từ lỗi sang đạt, trung bình: **28%** → **49%**; cũng nâng điểm **3 mô hình khác**.

Cách báo cáo đã đổi

Harness-Bench (05/2026): 106 tác vụ, 5.194 quỹ đạo thực thi (trajectory); báo năng lực theo **cặp mô hình–harness**.

Terminal-Bench 2.0 (89 tác vụ): mỗi hàng ghi **cả mô hình lẫn agent**.

Lưu ý: Cùng GPT-5.5 trên Terminal-Bench 2.0: harness A đạt 84,7 ± 2,1, Codex CLI đạt 82,2 ± 2,2. Khoảng tin cậy (confidence interval) chồng nhau, chưa chứng minh harness A tốt hơn.

Phục vụ luận điểm của chương

Kết quả đo gắn với cặp mô hình cộng harness, nên harness phải được đo như một biến độc lập.

Nguồn: Lewis, *Same Model, Different Harness*, arXiv:2608.26218 (26/08/2026); arXiv:2605.27922; arXiv:2601.11868; tbench/Harbor (04–05/2026).

Vòng lặp agent chỉ dừng khi có bằng chứng

Vòng lặp agent (agent loop): gọi công cụ, nhận kết quả, lặp lại đến khi xác minh xong việc.

```
def run(task, model, tools, policy, sandbox):
  ctx = assemble(SYSTEM, load_instruction_files(), task)          # 1
  for step in range(MAX_STEPS):                                # stop rule
    reply = model(ctx, tools.schemas())
    ctx.append(reply)                          # keep the tool-call request
    if not reply.tool_calls:
      if verify(task, sandbox): return reply          # done = proven
      ctx.append(user("Not done: tests fail")); continue
    for call in reply.tool_calls:
      decision = policy.check(call)                    # allow/ask/deny
      if decision == "ask": decision = human(call)
      obs = sandbox.run(call) if decision == "allow" \
          else denied(call)
      ctx.append(tool_result(call.id, truncate(obs))) # match call_id
    if tokens(ctx) > LIMIT: ctx = compact(ctx)                    # 6
    hooks.after_step(ctx)                                         # 8
  return escalate("budget exhausted")
```

Đọc theo số của sơ đồ

1: file hướng dẫn được nạp trước tác vụ.
`policy` quyết định lệnh có chạy không; `sandbox` giới hạn lệnh chạm được gì.
`append(reply)` rồi mới gắn kết quả theo `call.id`.
`truncate` trả 20 dòng liên quan, không trả 20.000.
`verify` buộc "xong" phải chứng minh được.

Lưu ý: Hết ngân sách thì báo lên con người, không trả lời như thể đã xong (lỗi "kết thúc sớm" của MAST).

Chỉ cưỡng chế mới ngăn được hành động

Chỉ dẫn (guidance) là chữ mô hình đọc; cưỡng chế (enforcement) là mã chạy ngoài mô hình.

Lớp	Ví dụ	Làm được	Không đảm bảo
File hướng dẫn	`CLAUDE.md`, `AGENTS.md`	báo quy ước, lệnh kiểm thử, vùng cấm	mô hình làm theo; chỉ là chữ trong ngữ cảnh
Quyền	cho phép, hỏi, từ chối theo lệnh hoặc đường dẫn	chặn trước khi chạy; buộc con người phê duyệt	lệnh được phép vẫn gây hại được
Sandbox	thư mục ghi giới hạn, chặn mạng	giới hạn hậu quả khi lệnh đã chạy	đúng ý định; cần chặn cả tệp lẫn mạng
Hook	mã chạy trước hoặc sau mỗi lời gọi công cụ	kiểm tra tất định: chặn tệp `.env`, biên dịch	bắt lỗi ngữ nghĩa

Lưu ý: Chế độ "tự duyệt mọi thứ" chỉ chấp nhận được bên trong sandbox độc lập. Ví dụ: `AGENTS.md` ghi "không sửa `generated/`", nhưng chỉ sandbox cho ghi trong `src/` mới ngăn được việc ghi.

Phục vụ luận điểm của chương

Một quy tắc chỉ bảo vệ được khi nằm ở lớp cưỡng chế, nên harness phải được rà soát như mã nguồn.

Nguồn: openai/codex `agents_md.rs`; Anthropic, *Claude Code sandboxing* (2025): cần cách ly cả hệ thống tệp lẫn mạng.

Công cụ là giao diện: đầu ra gọn, lỗi nêu cách sửa

Kém

`get_logs()`
trả 20.000 dòng
`Error 400`

Tốt

`search_logs(query, since, limit=20)`
trả 8 dòng khớp và 2 dòng ngữ cảnh
`start_date must be YYYY-MM-DD; got 2026/10/05`

Năm quy tắc thiết kế ACI

1. Tên và mô tả công cụ không chồng nhau (`search` và `find`?).
2. Tham số không mơ hồ: `user_id`, không phải `user`.
3. Đầu ra gọn: phân trang, cắt, tóm tắt.
4. Lỗi chỉ đường: nêu sai ở đâu và sửa thế nào.
5. Giữ lỗi trong ngữ cảnh để mô hình không lặp lại.

Đo, không đoán

SWE-agent (2024) đặt tên cho giao diện này là ACI. Viết 20 tác vụ thật có đáp án kiểm được, chạy agent, rồi đọc vết thực thi để thấy công cụ nào bị gọi sai, gọi thừa hoặc gọi lặp. Sửa mô tả công cụ trước khi sửa chỉ dẫn hệ thống (system prompt).

Nguồn: Yang et al., *SWE-agent: Agent-Computer Interfaces…*, arXiv:2405.15793; Anthropic, *Writing effective tools for agents* (11/09/2025); Manus, *Context Engineering for AI Agents* (2025).

Mười hai câu hỏi để rà soát một harness

Ngữ cảnh

- ☑ Phần đầu ngữ cảnh ổn định để bộ nhớ đệm (cache)?
- ☑ Đầu ra công cụ lớn được cắt hoặc lưu ra tệp?
- ☑ Có cách xử lý khi ngữ cảnh đầy?

Công cụ

- ☑ Mỗi công cụ một mục đích, tên không trùng?
- ☑ Lỗi công cụ chỉ ra cách sửa?
- ☑ Có 20+ tác vụ thật để đánh giá công cụ?

Quyền và nơi chạy

- ☑ Lệnh nào cho phép, hỏi, từ chối đã viết ra tệp?
- ☑ Sandbox chặn cả hệ thống tệp lẫn mạng?
- ☑ Thông tin bí mật (secret) không bao giờ vào ngữ cảnh?

Dừng và xác minh

- ☑ "Xong" nghĩa là kiểm thử tự động đạt?
- ☑ Có giới hạn bước, token, thời gian?
- ☑ Vết thực thi đủ để đọc lại từng quyết định?

Khi agent viết mã, kỹ sư viết gì?

OpenAI (02/2026) gọi việc này là "harness engineering": kỹ sư viết ý định kiểm được, giữ môi trường chạy được và mã hoá quy ước thành luật kiểm tự động. Dự án nội bộ, số liệu do công ty tự báo cáo, không có nhóm đối chứng.

Nguồn: OpenAI, *Harness engineering: leveraging Codex in an agent-first world* (11/02/2026).

Câu hỏi của chương: **DeepSeek Harness được thiết kế thế nào, hoạt động ra sao, và vì sao được chú ý?**

DeepSeek Harness (dsh) là harness mã nguồn mở dựng hoàn toàn từ plugin (mô-đun cắm thêm vào lõi hệ thống), nên vòng lặp agent, công cụ, sandbox, quyền và nén ngữ cảnh đều thay được. Hiện nó có giá trị chủ yếu như bản thiết kế tham chiếu để học, chưa sẵn sàng cho môi trường vận hành.

1. **Vì sao được chú ý.** Hãng làm mô hình tự phát hành harness, giấy phép MIT, khoảng 243,8 nghìn sao sau chưa đầy hai tháng; nhưng mới là bản xem trước, chưa kiểm toán.

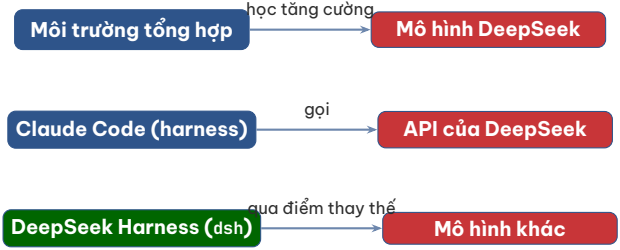
2. **Thiết kế.** Lõi Cordis chỉ ghép plugin; hồ sơ cấu hình chọn plugin nào chạy; 32 điểm thay thế (seam) cho phép đổi mô hình, sandbox, hệ tệp, nén ngữ cảnh, subagent.

3. **Cách hoạt động.** Mỗi lượt ghi vào nhật ký chỉ ghi thêm (append-only log); mỗi lời gọi công cụ qua hook, quyền, phê duyệt, sandbox; subagent có thể là Codex.

Sau chương này: so sánh dsh với harness khác, đọc được đường đi của một lời gọi công cụ, chọn ra thiết kế đáng mượn.

“DeepSeek harness” có ba nghĩa khác nhau

Khi một bài báo nói “X dùng harness của Y”, câu hỏi đầu tiên là: **thành phần nào** vừa đổi?



(1) Huấn luyện: DeepSeek-V3.2 học dùng công cụ trên nhiều môi trường agent được *tổng hợp*. Harness xuất hiện ở lúc huấn luyện.

(2) Đổi mô hình: điểm cuối (endpoint) tương thích Anthropic cho phép Claude Code chạy trên mô hình của DeepSeek.

(3) Đổi harness: sản phẩm mã nguồn mở (08/2026); mô hình chỉ là một plugin, subagent có thể là Codex hay Claude Code.

Vì sao cần tách

Ba cách nói trên đổi ba thành phần khác nhau: **trọng số**, **nhà cung cấp mô hình**, **phần mềm điều khiển**. So sánh công bằng chỉ đổi một thành phần mỗi lần, đúng như thí nghiệm ở chương 4. Phần còn lại của chương nói về nghĩa thứ ba.

Nguồn: DeepSeek-V3.2, arXiv:2512.02556; api-docs.deepseek.com (hướng dẫn tích hợp Claude Code); github.com/deepseek-ai/deepseek-harness (bản xem trước, tạo 13/08/2026).

Được chú ý vì ba lý do, kèm ba hạn chế

Ba lý do cộng đồng chú ý

1. Mô hình và harness cùng một hãng. Kết quả agent phụ thuộc cả mô hình lẫn harness (chương 4); hãng làm mô hình nay công bố cả harness chính thức.

2. Mở hoàn toàn. Giấy phép MIT; khoảng 243,8 nghìn sao và 29,2 nghìn fork (05/10/2026), repo tạo 13/08/2026. Số sao đo mức quan tâm, không đo chất lượng.

3. Thiết kế khác hướng. Vòng lặp agent và kho phiên cũng là plugin thay được; theo DataCamp (thứ cấp), Claude Code giữ vòng lặp cố định.

Lưu ý: 1. Bản xem trước. README ghi: sẽ có thay đổi phá vỡ tương thích. Mọi chi tiết cài đặt trong chương này có thể đã đổi.

2. Chưa kiểm toán bảo mật. `SAFETY.md`: không được coi là an toàn hay sẵn sàng cho môi trường vận hành; sandbox giảm rủi ro nhưng không bảo đảm cách ly.

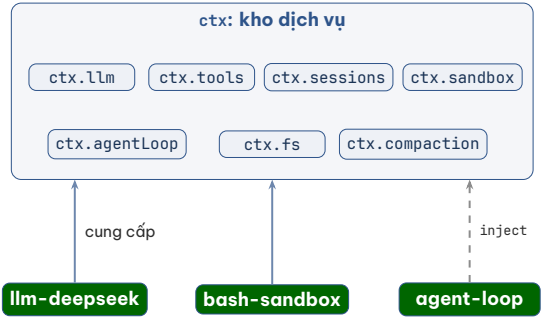
3. Chưa có số đo năng lực. Các trang đã đọc không nêu điểm SWE-bench hay Terminal-Bench; thư mục `benchmarks/` chỉ đo hiệu năng nội bộ. Chưa có cơ sở xếp hạng với Claude Code hay Codex.

Cách đọc đúng

Chú ý đến dsh là chú ý đến **cách thiết kế**, không phải thứ hạng.

Nguồn: deepseek-harness (README, `SAFETY.md`, `benchmarks/`); DataCamp, *What Is DeepSeek Harness?*; 05/10/2026.

Mọi thành phần là plugin, Cordis ghép chúng lại



Plugin là dịch vụ, phụ thuộc khai báo

Mỗi dịch vụ giữ một khoá cố định (`ctx.tools`, `ctx.llm`). Plugin khai báo dịch vụ cần dùng bằng `inject` và chờ chúng xuất hiện; không viết thứ tự nạp.

Sự kiện có kiểu

Năm chế độ phát: `emit`, `waterfall`, `parallel`, `serial`, `bail`. Chế độ `waterfall` là nền của hook trước và sau khi chạy công cụ.

Đăng ký đảo ngược được

Công cụ, mục chỉ dẫn, trình nghe sự kiện cài qua `ctx.effect()`; gỡ plugin thì chúng gỡ theo.

Hệ quả thiết kế

Không có lõi đặc quyền để vá: muốn đổi hành vi, gắn thêm một plugin bên cạnh các plugin khác (“everything is a plugin”).

Nguồn: deepseek-harness: `docs/cordis-primer.md`, `docs/architecture.md`; bài báo Cordis: *A Programming Paradigm for Spatiotemporal Composability*.

Một bản dsh là phép ghép ba lớp cấu hình



Dịch vụ	Vai trò
`ctx.sessions`	nhật ký sự kiện chỉ ghi thêm, kho trong bộ nhớ
`ctx.systemPrompt`	lắp chỉ dẫn hệ thống và lược đồ công cụ
`ctx.tools`	kho công cụ và chuỗi thực thi
`ctx.agentLoop`	vòng lặp mặc định của agent
`ctx.llm`	luồng mô hình, bộ chuyển đổi nhà cung cấp

```
# in cay plugin da ghep cua mot profile
dsh --profile web --dump-config
# mo giao dien web o 127.0.0.1:3080
npx @deepseek-ai/dsh web
```

Đổi một thành phần

Ghi một patch thay đúng hàng cấu hình đó; không sửa mã lõi. Patch của người dùng áp sau cùng nên thắng patch của bundle.

Nguồn: deepseek-harness: `docs/architecture.md` (profile `web`, `headless`, `sdk`, `sdk-minimal`, `acp`; thứ tự patch); README (lệnh khởi chạy); truy cập 05/10/2026.

32 điểm thay thế cho phép đổi từng thành phần

Điểm thay thế	Nhà cung cấp có sẵn	Đổi để
`ctx.llm`	`llm-deepseek`, `llm-pi-ai`, `llm-replay`	đổi nhà cung cấp mô hình
`ctx.sandbox`	`sandbox-local`, `sandbox-ssh`	cách ly cục bộ hoặc từ xa
`ctx.fs`	`fs-local`, `fs-sandbox`, `fs-ssh`	đổi nơi đọc và ghi tệp
`ctx.shell`	`bash-local`, `bash-sandbox`, `pwsh-local`	đổi vỏ lệnh
`ctx.compaction`	`compaction-basic`	đổi cách nén ngữ cảnh
`ctx.subagents`	nhiều provider (frame sau)	đổi nơi chạy subagent
`ctx.approval`	nhà cung cấp phê duyệt	đổi cách hỏi người dùng

Điểm thay thế (seam) — một dịch vụ có *định nghĩa* (giao diện), một hoặc nhiều *nhà cung cấp* và *bên dùng*. Bên dùng chỉ biết giao diện.

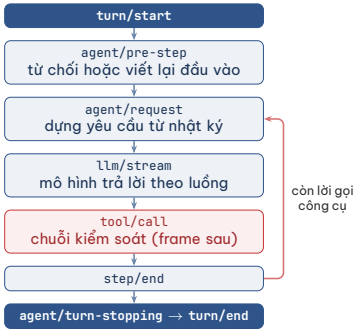
Ví dụ trong tài liệu

Trỏ hệ tệp và tiến trình con tới cùng một sandbox từ xa, thì Bash, giao diện dòng lệnh giả (PTY) và máy chủ ngôn ngữ (LSP) chạy theo vào đó, không phải viết lại nhà cung cấp nào.

Lưu ý: Nhiều điểm thay thế nghĩa là nhiều thứ phải hiểu: phiên bản của từng plugin phải tương thích nhau. Đây là cái giá của tính linh hoạt.

Nguồn: deepseek-harness: `docs/capability-seams.md` (32 điểm thay thế mang vai trò `seam`), `docs/architecture.md`; truy cập 05/10/2026.

Mô hình chỉ thấy những gì đã ghi vào nhật ký



Nguyên tắc: thấy được nghĩa là đã ghi

Mỗi yêu cầu gửi cho mô hình phải dựng lại được từ nhật ký phiên làm việc (JSONL có phiên bản, chỉ ghi thêm). Mọi sự kiện được chấp nhận phải vào nhật ký *trước* khi bước kế tiếp được dựng từ nó.

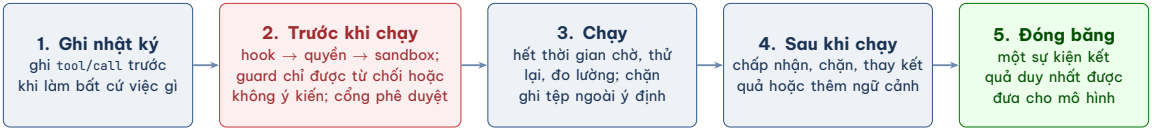
Ba hệ quả

Khôi phục: đọc lại nhật ký để tiếp tục một phiên.
Phát lại (replay): cùng nhật ký cho ra cùng trạng thái, dùng để gỡ lỗi và kiểm thử.
Hai miền sự kiện: `session/*` bền vững; `agent/*` chỉ điều phối thời gian thực.

Lưu ý: So với vòng lặp ở chương 4, dsh thêm: nhật ký bền vững; lời gọi *song song được* (mặc định tối đa 10) và *độc quyền* (chạy một mình, làm rào chắn thứ tự); huỷ hợp tác.

Nguồn: deepseek-harness: `docs/architecture.md`, `packages/core/agent-loop`, `docs/agent-lifecycle.md`; truy cập 05/10/2026.

Lời gọi công cụ qua năm chặng trước khi có kết quả



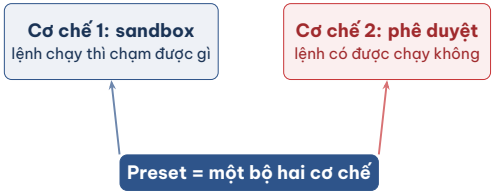
Ví dụ (minh hoạ)	Diễn biến	Kết quả
chạy kiểm thử trong thư mục làm việc	quyền cho phép, chạy trong sandbox	8 dòng kết quả
đẩy mã lên kho từ xa	quyền là hỏi, dừng ở cổng phê duyệt	chờ người duyệt
như trên, không ai trả lời	cổng phê duyệt vắng mặt	từ chối

Mỗi lời gọi đều có một kết quả

Nếu lỗi xảy ra giữa chừng, vòng lặp tạo kết quả tổng hợp `TOOL_OUTCOME_UNKNOWN` (đã ghi lời gọi nhưng chưa có kết quả) hoặc `TOOL_NOT_STARTED`, thay vì để hội thoại mất cấu trúc.

Nguồn: deepseek-harness: `docs/tool-execution-pipeline.md` (15 trạm, gom thành 5 chặng). Bảng ví dụ do giảng viên dựng để minh hoạ, không phải đầu ra thật của dsh.

Quyền và sandbox là hai cơ chế độc lập



lớp preset không sở hữu chính sách thực thi

Preset mặc định	Sandbox	Phê duyệt
`workspace-write`	`workspace-write`	`ask`
`danger-full-access`	`danger-full-access`	`never`

Mặc định từ chối khi thiếu thông tin (fail-closed)

Cổng phê duyệt vắng mặt hoặc không trả lời được thì kết quả là từ chối. Không có cách bỏ qua phê duyệt mà không để lại dấu vết.

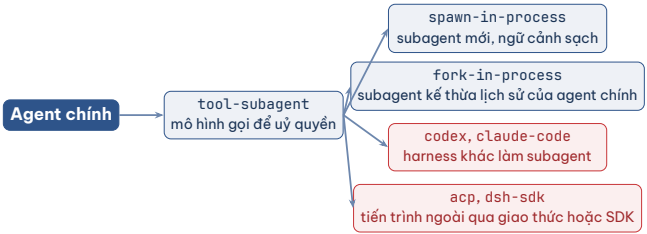
Lưu ý: Preset `danger-full-access` chỉ chấp nhận được trong môi trường dùng một lần. `SAFETY.md` khuyến nghị quyền tối thiểu, máy ảo hoặc container dùng một lần, sao lưu, không để lộ thông tin bí mật. Các lớp này *giảm* rủi ro, không bảo đảm cách ly.

Nối với chương 4

Đây là "chỉ dẫn không phải cưỡng chế" được cài thành mã: quyền chặn *trước* khi chạy, sandbox giới hạn *hậu quả* sau khi chạy; hai lớp độc lập.

Nguồn: deepseek-harness: `docs/subsystems/permission-presets.md`, `SAFETY.md`; 05/10/2026.

Subagent có thể là dsh, Codex hoặc Claude Code



Spawn khác fork ở ngữ cảnh

`spawn`: con khởi động với ngữ cảnh trống, cha chỉ nhận kết quả (lớp cách ly ngữ cảnh, chương 6). `fork`: con mang theo lịch sử của cha, đủ bối cảnh nhưng tốn token hơn.

Ý nghĩa

dsh có thể làm agent điều phối, còn Codex hoặc Claude Code làm agent thực thi. Subagent nhận thêm lượt sau khi trả kết quả; cha nhắn tin, ngắt, liệt kê qua `tool-subagent-control`.

Nguồn: deepseek-harness: `packages/subagent/README.md`; 05/10/2026. Hệ quả về token của `fork` là suy luận từ định nghĩa "seeded from the parent's completed history".

dsh ở đầu linh hoạt của phổ thiết kế



Đây là đánh đổi, không phải xếp hạng

Càng nhiều điểm thay thế, càng nhiều thứ để cấu hình, kiểm thử và cấp quyền. Theo DataCamp (thứ cấp), dsh có chế độ *Minimal* chỉ hai công cụ (shell, trình sửa chuỗi) để đo mô hình trên harness tối giản.

Lưu ý: Tính năng đổi theo tháng. Pi và dsh đều ghi rõ sandbox là trách nhiệm người vận hành; dsh là bản xem trước. Không dùng sơ đồ này để chọn sản phẩm.

Nguồn: README các repo (05/10/2026): mini-swe-agent, earendil-works/pi, openai/codex, deepseek-harness; tài liệu Claude Code; DataCamp.

Năm thiết kế đáng mượn từ dsh

Năm thiết kế đáng mượn cho harness của riêng mình

1. **Nhật ký chỉ ghi thêm là nguồn sự thật.** Mọi thứ mô hình thấy dựng lại được từ nhật ký; từ đó có khôi phục, phát lại, truy vết.
2. **Mặc định từ chối khi thiếu thông tin.** Phê duyệt vắng mặt nghĩa là từ chối.
3. **Tách quyền khỏi sandbox.** Quyền quyết định lệnh có chạy không; sandbox giới hạn lệnh chạm được gì.
4. **Mỗi thành phần đứng sau một điểm thay thế có tên.** Đổi mô hình, sandbox hay cách nén ngữ cảnh mà không sửa phần còn lại.
5. **Chọn loại subagent theo nhu cầu cách ly ngữ cảnh.** Con mới khi cần cách ly; con thừa kế lịch sử khi cần bối cảnh.

Lưu ý: Chưa nên: dùng cho môi trường vận hành (chưa kiểm toán, API còn đổi); kết luận về năng lực so với sản phẩm khác (chưa có số đo công bố); coi số sao là bằng chứng chất lượng.

Bài tập

Chấm dsh theo checklist 12 mục ở chương 4. Với mỗi mục, ghi dsh đáp ứng bằng thiết kế nào (tên package hoặc dịch vụ) hay tài liệu chưa nêu. Mục "chưa nêu" là câu hỏi cần đặt trước khi chọn nó.

Nguồn: Tổng hợp từ các frame trước trong chương; deepseek-harness: `docs/architecture.md`, `docs/defensive-patterns.md`, `SAFETY.md`.

Chương 6 — Thiết kế ngữ cảnh giữa các agent

Câu hỏi của chương: **Cái gì được chuyển giữa các agent, cái gì bị cắt, cái gì còn lại sau khi đặt lại ngữ cảnh?**

Giữa các agent, ngữ cảnh đi qua hai kênh là brief (bản mô tả việc giao cho subagent) và kết quả trả về, còn dữ liệu lớn đi qua tệp. Thiết kế ngữ cảnh (context engineering) là quyết định cái gì được chuyển, cái gì bị cắt và cái gì còn lại sau khi ngữ cảnh bị đặt lại.

1. **Hai kênh chính qua ranh giới.** Brief quyết định subagent làm đúng hay sai; 5 nhánh đọc 300k token, agent chính chỉ nhận 7,5k.

2. **Cắt những gì chưa cần.** Skill (gói hướng dẫn nạp khi cần) nạp theo nhu cầu (progressive disclosure): 50 skill tốn ∼5k token, không phải 150k.

3. **Chỉ trạng thái (state) ngoài ngữ cảnh sống sót.** Sau khi đặt lại ngữ cảnh (context reset), chỉ tệp ghi chú, git và kiểm thử (testing) còn nguyên; bản nén giữ một phần.

Sau chương này: Viết brief đủ chín trường, tính chi phí ngữ cảnh của một subagent, và chọn trạng thái nào phải ghi ra ngoài ngữ cảnh.

Brief là toàn bộ ngữ cảnh mà subagent nhận được

Thiếu các quyết định đã có, subagent hiểu sai: brief là **ngữ cảnh được đóng gói có chủ đích**.

```
BRIEF (lead → subagent)
GOAL: one sentence, testable
WHY: what the lead will do with it                    # intent
DECIDED: choices already made - do not revisit        # no drift
SCOPE: in / out (who covers the rest)                 # no overlap
TOOLS: allowed tools + budget (15 calls)
SOURCES: prefer primary; list URLs used
RETURN: JSON schema; ≤ 300 words                      # compress
 + "surprises" + "open questions"                     # no withholding
ARTIFACTS: write long data to files, return paths
STOP: when GOAL met or budget spent                   # termination
```

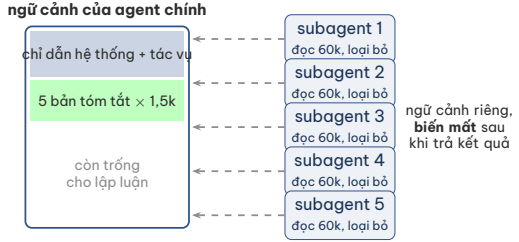
Mỗi trường chặn một kiểu lỗi MAST

`DECIDED`: lệch quyết định giữa các agent.
`SCOPE`: làm trùng việc.
`RETURN`: kết quả không theo đặc tả.
`surprises`: giữ lại thông tin.
`STOP`: không biết khi nào dừng.

Phục vụ luận điểm 1

Brief là một trong hai thứ duy nhất đi qua ranh giới, nên mọi điều subagent cần biết phải nằm trong đó. Nếu người đọc lạ phải đoán, subagent cũng phải đoán.

Subagent đọc 300k token, agent chính chỉ nhận 7,5k



Lớp cách ly ngữ cảnh

Subagent là lớp cách ly ngữ cảnh (context firewall): 5 nhánh $\times$ 60k token đọc = 300k token, nhưng agent chính chỉ nhận **7,5k** ($5 \times 1{,}5$k). Không có subagent, 300k token vượt cửa sổ ngữ cảnh.

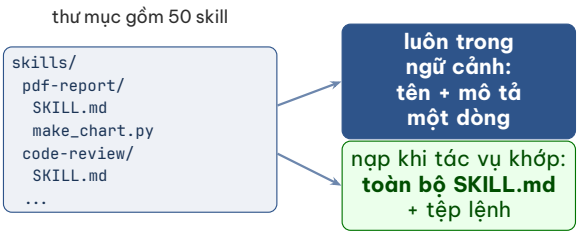
Đánh đổi

Bản tóm tắt làm mất chi tiết; khi cần, subagent ghi chi tiết ra tệp và trả về **đường dẫn**. Subagent hợp với việc đọc (nén được), không hợp với việc ghi: quyết định thiết kế bị nén là mất.

Phục vụ luận điểm 1

Chỉ kết quả đã nén đi qua ranh giới; chi tiết nào cần giữ phải được ghi ra ngoài ngữ cảnh.

Nạp skill theo nhu cầu: 5k token thay vì 150k



Phép tính (giả định minh hoạ)

50 skill × ∼100 token mô tả = **5k** token thường trực.
Nạp hết phần thân: 50 × 3k = **150k**.
Mỗi tác vụ thường cần 1–2 skill ⇒ ∼**11k**.

Vì sao thuộc chương này

Skill là quy trình làm việc dùng chung: subagent ghi lại cách làm đã chạy được, agent khác nạp lại. Chương 8 cho agent tự viết skill.

Phục vụ luận điểm 2

Chi phí thường trực chỉ gồm phần mô tả; mọi thứ chưa cần ở lượt hiện tại bị cắt khỏi ngữ cảnh.

Nguồn: Anthropic, *Introducing Agent Skills* (16/10/2025); agentskills.io (18/12/2025); tài liệu Pi.

Chỉ trạng thái ngoài ngữ cảnh sống sót khi đặt lại

Lớp trạng thái	Giữ cái gì	Sống sau đặt lại?
Lịch sử hội thoại	mọi lượt, mọi kết quả công cụ	không
Bản nén ngữ cảnh	tóm tắt để chạy tiếp	một phần: *mất chi tiết*
Tệp ghi chú, tiến độ	quyết định, việc còn lại	có
Git `commit`	mã đã chạy được	có, quay lui được
Kiểm thử	định nghĩa "xong"	có, **máy kiểm được**

Bốn khái niệm hay bị dùng lẫn

Nén ngữ cảnh giảm kích thước lịch sử.
Bộ nhớ (memory) lưu thông tin bền vững.
Điểm lưu (checkpoint) lưu trạng thái làm việc để khôi phục.
Xác minh chứng minh kết quả đúng.

Lưu ý: Anthropic ghi lại: chỉ dựa vào nén ngữ cảnh, agent dựng ứng dụng lớn vẫn thất bại vì phiên sau không biết phiên trước dừng ở đâu.

Phục vụ luận điểm 3

Thứ phải sống lâu hơn một cửa sổ ngữ cảnh thì phải nằm ngoài cửa sổ đó: tệp, git, kiểm thử.

Nguồn: Anthropic, *Effective harnesses for long-running agents* (26/11/2025); OpenAI, *Unrolling the Codex agent loop* (2026).

Bộ nhớ đệm và trần tài nguyên quyết định chi phí

Phép tính bộ nhớ đệm (giá 2025)

Manus (2025): đầu vào dài gấp ∼100 lần đầu ra, nên chi phí do **bộ nhớ đệm KV (KV cache)** quyết định. Lượt 10.000 token, 9.900 token là tiền tố lặp lại; giá 2025 là \$0,30 (có bộ nhớ đệm) và \$3 (không) mỗi triệu token. Trúng: ≈ **\$0,0033**. Trượt: **\$0,030**, gấp ∼9 lần.

Sản phẩm 2026	Ngữ cảnh agent thực thi	Trần tài nguyên
Claude Code Dynamic Workflows	riêng; chỉ kết quả cuối về phiên chính	16 đồng thời; 1.000 mỗi lần chạy
Claude Managed Agents	riêng, chung sandbox	≤ 20 agent, **chỉ 1 tầng**
OpenAI Responses	riêng; sinh được cây con	**không giới hạn độ sâu**

Lưu ý: Cả ba còn ở bản beta hoặc tài liệu mới (10/2026); giới hạn có thể đổi trong vài tháng.

Phục vụ luận điểm 2 và 3

Hạ tầng cũng cách ly ngữ cảnh agent thực thi và đặt trần ngay trong cấu hình.

Nguồn: Manus, *Context Engineering for AI Agents* (2025); code.claude.com/docs/en/workflows; platform.claude.com/docs/en/managed-agents; developers.openai.com/api/docs/guides/responses-multi-agent (05/10/2026).

Câu hỏi của chương: **Điều gì giúp agent làm việc hàng giờ mà không lạc mục tiêu?**

Tác vụ càng dài thì lỗi tích luỹ và trôi mục tiêu càng nặng; agent chạy được hàng giờ khi harness giữ trạng thái ngoài mô hình và quyết định dừng bằng cổng kiểm tra, không bằng lời của mô hình.

1. **Tác vụ dài hạn (long-horizon).** Độ dài tác vụ (time horizon) gấp đôi sau 7 tháng; 318 bước, mỗi bước đúng 99%, còn ∼4%; trôi mục tiêu (goal drift) có bốn nguồn.

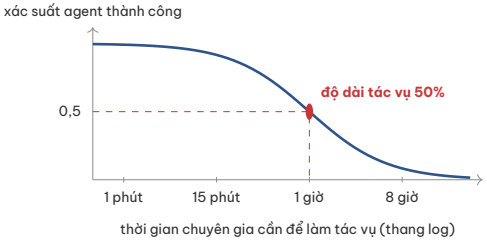
2. **Trạng thái ngoài mô hình.** Danh sách tính năng, tiến độ và git giữ việc qua nhiều phiên; 16 agent viết trình biên dịch C (∼100.000 dòng Rust) nhờ khoá và bộ kiểm thử có sẵn.

3. **Dừng bằng cổng.** Vòng lặp kết thúc khi qua cổng kiểm soát chất lượng (quality gate) máy kiểm được hoặc hết ngân sách; số đo chỉ so sánh được khi cùng hệ thống.

Sau chương này: Đọc đúng con số về độ dài tác vụ, thiết kế harness chạy nhiều phiên, và đặt cổng cùng điều kiện dừng cho vòng lặp.

Độ dài tác vụ agent làm được gấp đôi sau 7 tháng

Độ dài tác vụ 50% (METR): *thời gian chuyên gia người* cần cho việc mà agent làm xong với xác suất 50%.



Xu hướng và ngoại suy

2019–2025: gấp đôi khoảng mỗi 7 tháng; Claude 3.7 Sonnet (đầu 2025) $\approx$ **50 phút**; 2026: bộ tác vụ **TH1.1**. Ngoại suy 21 tháng ($\times$8): $\sim$6,7 giờ, là **mức sàn** vì 2024–2025 gấp đôi nhanh hơn ($\sim$4 tháng).

Lưu ý: Không phải thời gian agent chạy, cũng không có nghĩa agent làm 50% *mọi* việc dài 2 giờ.

Phục vụ luận điểm 1

Tác vụ 2 giờ-người cần hàng trăm hành động (OSWorld 2.0: 318 lời gọi công cụ); nếu mỗi bước đúng 99%, $0{,}99^{318} \approx$ **4%** nếu không tự sửa.

Nguồn: Kwa et al. (METR), arXiv:2503.14499 (18/03/2025); metr.org/time-horizons (cập nhật 05/2026); OSWorld 2.0, arXiv:2606.29537.

Bốn nguồn trôi mục tiêu, mỗi nguồn một cơ chế chặn

Nguồn	Dấu hiệu trong vết thực thi	Cơ chế của harness
Suy giảm ngữ cảnh (context rot)	trả lời câu hỏi đã giải quyết 3 giờ trước	nén có chủ đích; ghi quyết định ra tệp; khởi động lại agent với ngữ cảnh sạch
Bám theo nhận xét gần nhất	sửa đúng nhận xét rà soát cuối, trái đặc tả gốc	đọc lại đặc tả và `todo.md` sau mỗi N bước; đặc tả nằm trong tệp, không chỉ trong câu lệnh đầu
Tuyên bố xong sớm (premature victory)	"mọi tính năng đã hoàn thành" khi 40/200 còn lỗi	"xong" = danh sách tính năng + kiểm thử; agent chỉ được sửa trường `passes`
Tranh chấp giữa các agent	nhiều agent sửa cùng một bộ phân tích cú pháp, PR xung đột rồi bị bỏ	chia theo mô-đun; cổng tích hợp (integration gate): chỉ một điểm hợp nhất, qua kiểm thử mới vào nhánh chính

Chuyển sang luận điểm 2

Mọi cơ chế đều chuyển một thứ từ **trí nhớ của mô hình** sang **trạng thái bên ngoài, kiểm được**: tệp, git, kiểm thử, khoá.

Chạy nhiều phiên nhờ danh sách tính năng và git




```
{ "id": "chat-send-message",
  "steps": ["open app", "type hi",
      "see streamed reply"],
  "passes": false }        # only field agent edits
```

Ba quyết định đáng áp dụng

JSON thay vì Markdown: mô hình ít tự ý viết lại JSON hơn.
Một tính năng mỗi phiên: không làm cả ứng dụng một lần.
Cuối phiên ở trạng thái sạch: hợp nhất được, có ghi chú.

Phục vụ luận điểm 2

Mỗi phiên bắt đầu với ngữ cảnh sạch; tiến độ nằm trong tệp, git và danh sách tính năng, không nằm trong mô hình.

Nguồn: Anthropic, *Effective harnesses for long-running agents* (26/11/2025): ví dụ ứng dụng chat, hơn 200 tính năng ban đầu đều `passes: false`.

Nhiều agent cần khoá nhận tác vụ hoặc tách vai

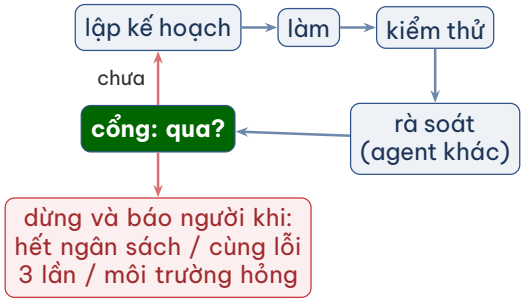
	Trình biên dịch C (Anthropic)	Trình duyệt (Cursor)
Quy mô	16 agent song song, ∼2.000 phiên, ∼$20.000 API; ∼100.000 dòng Rust	trình duyệt từ đầu: gần 1 tuần, >1 triệu dòng, 1.000 tệp (công ty tự báo cáo)
Phối hợp	khoá nhận tác vụ (task-claim lock) bằng tệp trong `current_tasks/`; mỗi agent một bản sao, kéo–hợp nhất–đẩy	lần 1 ngang hàng + khoá: agent chọn việc nhỏ, né việc khó. Lần 2 tách vai: lập kế hoạch, lập kế hoạch con, thực thi, và một agent phán quyết có lặp tiếp không
Vì sao chạy được	có tiêu chí đúng/sai tự động (bộ kiểm thử có sẵn); chia được thành phần ít đụng nhau	viết lại `scratchpad.md` thay vì nối thêm; khởi động lại agent định kỳ
Giới hạn ghi nhận	chưa dùng được ở môi trường vận hành: thiếu x86 16-bit để khởi động, còn dùng assembler và linker của GCC	agent ngủ, dừng sớm, không hợp nhất, và *tuyên bố đã xong khi chưa xong*

Phục vụ luận điểm 2

Cả hai ca đưa việc nhận tác vụ và tiêu chí "xong" ra ngoài mô hình; Cognition nhận xét các minh hoạ lớn kiểu này có tiêu chí thành công đơn giản, kiểm được.

Nguồn: Anthropic, *Building a C compiler with a team of parallel Claudes* (05/02/2026); Cursor, *Scaling long-running autonomous coding* (14/01/2026), *Towards self-driving codebases* (05/02/2026); Cognition (22/04/2026).

Vòng lặp dừng khi qua cổng kiểm soát chất lượng



Vòng lặp cho tới khi qua cổng

Cộng đồng gọi mẫu này là vòng lặp "Ralph Wiggum"; OpenAI dùng cho vòng rà soát của Codex.

Điều làm vòng lặp an toàn

Cổng phải do máy kiểm được: kiểm thử, linter, agent rà soát độc lập. **Điều kiện dừng** viết trước, kể cả dừng vì *thất bại*.

Lưu ý: Thử lại 50 lần khi thiếu thư viện chỉ tốn token: phân biệt lỗi mã với lỗi môi trường.

Phục vụ luận điểm 3

Quyết định dừng thuộc về cổng và ngân sách, không thuộc về lời tuyên bố "xong" của mô hình.

Nguồn: OpenAI, *Harness engineering* (11/02/2026).

Chỉ so sánh số đo dài hạn khi cùng một hệ thống

Bộ đánh giá chuẩn	Đo gì	Lưu ý khi đọc
METR (độ dài tác vụ)	độ dài việc làm được ở 50% và 80%	phụ thuộc harness; TH1.0 $\neq$ TH1.1
SWE-bench Verified / Pro	sửa lỗi GitHub thật (500 tác vụ / bản khó hơn)	Pro: mô hình dưới 25% khi ra mắt (09/2025); Verified đã gần bão hoà (saturation)
Terminal-Bench 2.0	89 tác vụ dòng lệnh thực tế	mỗi điểm là **mô hình + agent**; đọc khoảng tin cậy (confidence interval)
OSWorld 2.0	dùng máy tính, $\sim$1,6 giờ mỗi tác vụ	không trộn tác vụ, ảnh máy ảo giữa các bản phát hành
τ^2-bench	hội thoại có công cụ + người dùng mô phỏng	cần cùng cấu hình người dùng mô phỏng
Vending-Bench 2 / Arena	kinh doanh mô phỏng dài hạn; bản Arena nhiều agent cạnh tranh	đo bằng tiền cuối kỳ, không phải đạt/không đạt

Câu hỏi bắt buộc trước mọi so sánh

Cùng mô hình, harness, phiên bản bộ đánh giá, ngân sách và bộ chấm? Một câu trả lời "không" nghĩa là đó là so sánh *hệ thống*, không phải so sánh *mô hình*.

Nguồn: swebench.com; SWE-Bench Pro arXiv:2509.16941; Terminal-Bench arXiv:2601.11868; OSWorld 2.0 arXiv:2606.29537; tau2-bench; andonlabs.com (Vending-Bench).

Câu hỏi của chương: **Agent "tự tiến hoá" thực chất thay đổi cái gì, và khi nào thay đổi đó đáng giữ?**

Agent tự tiến hoá (self-evolving) thường không sửa trọng số mà sửa một thành phần bên ngoài mô hình (ngữ cảnh, bộ nhớ, skill, mã); một thay đổi chỉ đáng giữ khi vượt kiểm tra trên dữ liệu agent không đọc được, và điểm báo cáo lấy từ một tập held-out riêng, vì reward hacking là rủi ro chính.

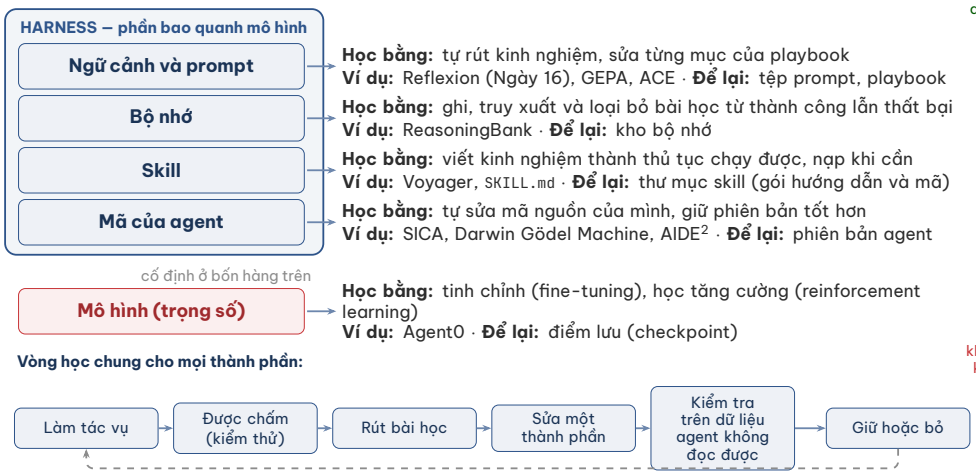
1. **Thứ thay đổi.** ACE chỉ sửa playbook (danh sách quy tắc agent tự tích luỹ), giữ nguyên mô hình, báo cáo +10,6% trên tác vụ agent.

2. **Tự sửa mã.** Darwin Gödel Machine nâng SWE-bench từ 20,0% lên 50,0% sau 80 vòng; lợi ích của bản cập nhật phụ thuộc năng lực mô hình nền.

3. **Khi nào đáng giữ.** Reward hacking là agent tối ưu chỉ số đo thay vì mục tiêu thật; chỉ giữ thay đổi vượt kiểm tra trên dữ liệu agent không đọc được; điểm báo cáo lấy từ held-out (tập dữ liệu giữ riêng, không dùng để chọn thay đổi).

Sau chương này: áp dụng vòng rút bài học (lesson loop) an toàn cho agent của mình.

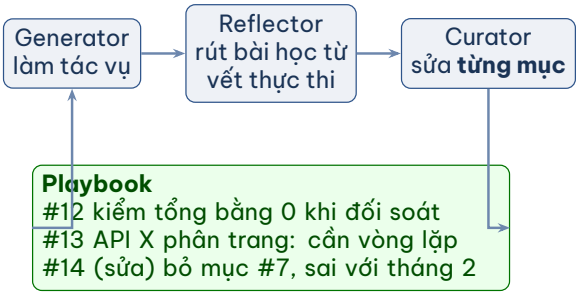
Năm thành phần có thể tiến hoá, từ dễ gỡ đến khó gỡ



Chỉ hàng cuối đụng đến trọng số: Agent0 tăng Qwen3-8B-Base **18%** ở toán và **24%** ở suy luận tổng quát.

Nguồn: Khảo sát: arXiv:2507.21046; arXiv:2508.07407; arXiv:2607.13104 (07/2026). Agent0, arXiv:2511.16043. Các hệ thống nêu ở frame sau.

ACE: sửa từng mục của playbook, mô hình đóng băng



mô hình **đóng băng**; chỉ playbook thay đổi

ACE: cập nhật playbook theo mục

Sửa **từng mục nhỏ** thay vì viết lại cả prompt, để tránh thiên lệch tóm gọn (brevity bias) và sụp đổ ngữ cảnh (context collapse). **+10,6%** trên tác vụ agent, **+8,6%** trên tác vụ tài chính, không cần nhãn.

GEPA: tự rút kinh nghiệm để tiến hoá prompt

Đọc vết thực thi, chẩn đoán bằng ngôn ngữ, đề xuất prompt mới, giữ tập phương án Pareto. So với GRPO: hơn **6%** trung bình (tối đa 20%), số lượt chạy thử (rollout) **ít hơn tới 35 lần**.

Phục vụ luận điểm 1

Ngữ cảnh thay đổi theo từng mục, trọng số giữ nguyên; mỗi mục xem lại và gỡ được.

Nguồn: Zhang et al., *ACE*, arXiv:2510.04618 (ICLR 2026); Agrawal et al., *GEPA*, arXiv:2507.19457 (ICLR 2026). Số liệu theo bản preprint.

Bài học dạng mã chạy được đáng tin hơn ghi chú

ReasoningBank

Chưng cất chiến lược từ **cả thành công lẫn thất bại** do agent tự đánh giá; truy xuất khi gặp tác vụ giống.

Ví dụ bài học: không tin công cụ đã chạy chỉ vì agent vừa *nói* nó đã chạy.

Voyager

Agent Minecraft tích luỹ **thư viện skill dạng mã chạy được**, không tinh chỉnh mô hình.

Số vật phẩm khác nhau gấp $3{,}3$ lần; đạt mốc công nghệ nhanh hơn tới $15{,}3$ lần.

Agent tự viết skill

Skill (`SKILL.md` cùng tập lệnh) là bộ nhớ thủ tục **chạy được và duyệt lại được**; bài học thành hiện vật người đọc được.

Lưu ý: Bộ nhớ do agent tự chấm có thể lưu **bài học sai** vĩnh viễn. Mỗi mục cần hạn sử dụng, nguồn gốc, và kiểm lại trên tập tác vụ held-out.

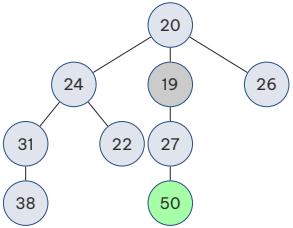
Phục vụ luận điểm 1

`mine_wood()` gọi lại và kiểm tra được; ghi chú "nhớ chặt gỗ trước" thì mô hình có thể đọc rồi bỏ qua.

Nguồn: ReasoningBank, arXiv:2509.25140 (ICLR 2026); Wang et al., *Voyager*, arXiv:2305.16291; Anthropic, *Introducing Agent Skills* (16/10/2025).

Tự sửa mã nâng SWE-bench từ 20,0% lên 50,0%

Bản sửa được nhận trở thành agent sửa ở vòng sau: tự cải tiến đệ quy (recursive self-improvement).



Nhánh 19% (xám) kém cha nhưng sinh ra con 50%. (minh hoạ, số giả định)

Hệ thống	Thứ được sửa	Kết quả báo cáo
Darwin Gödel Machine (Sakana)	mã của agent lập trình	SWE-bench **20,0%** → **50,0%**; Polyglot 14,2% → 30,7%
SICA	mã của chính agent	SWE-bench Verified (tập con) **17%** → **53%**
AlphaEvolve (DeepMind)	thuật toán	cần bộ chấm tự động tin được
AIDE^2 (**preprint**, 22/09/2026)	mã của agent nghiên cứu	8 ngày, 7 cải tiến nối tiếp; 4/4 held-out bằng hoặc hơn agent người thiết kế; reward hacking **55%** → **32%**

Phục vụ luận điểm 2

Mỗi kết quả cần bộ chấm tin được; AIDE^2 là preprint, chưa phải năng lực tổng quát.

Nguồn: arXiv:2505.22954 (DGM); arXiv:2504.15228 (SICA); DeepMind blog (14/05/2025; cập nhật 07/05/2026); arXiv:2609.26457 (AIDE^2, abstract).

Cập nhật harness không đồng nghĩa với có lợi

Nghiên cứu 05/2026 tách hai năng lực: viết bản cập nhật và hưởng lợi từ nó.



Hình dạng định tính theo abstract, không phải số đo.

Kết quả

Viết cập nhật: phẳng. Bản của Qwen3.5-9B cho lợi ích tương đương bản của Claude Opus 4.6.

Hưởng lợi: không đơn điệu. Mô hình yếu ít lợi, trung bình lợi nhất, mạnh lợi ít hơn mức trung bình.

Mô hình yếu thất bại vì **không kích hoạt** skill, hoặc kích hoạt mà **không làm theo**.

Hệ quả thiết kế

Dùng mô hình rẻ để *viết* bài học, dồn ngân sách vào agent *làm* tác vụ, đo cả **tỷ lệ sử dụng** skill.

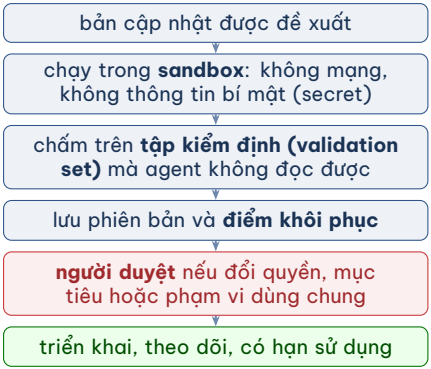
Nguồn: *Harness Updating Is Not Harness Benefit*, arXiv:2605.30621 (28/05/2026). Liên quan: *Self-Harness*, arXiv:2606.09498 (06/2026).

Agent thấy hàm chấm sẽ tối ưu hàm chấm

Reward hacking — tăng điểm của hàm đo mà không đạt mục tiêu mà hàm đo đại diện.

Ví dụ từ Darwin Gödel Machine

Trong một thí nghiệm phát hiện ảo giác, agent tự sửa **tối ưu điểm của bộ phát hiện** thay vì giảm ảo giác thật. Hiện tượng xảy ra nhiều hơn khi hàm kiểm **nhìn thấy được** với agent.



Phục vụ luận điểm 3

Chỉ thay đổi vượt kiểm tra trên held-out mà agent không đọc được mới được giữ.

Nguồn: arXiv:2505.22954 (DGM, mục objective hacking); kho SICA khuyến nghị chạy trong Docker.

Vòng rút bài học an toàn: mỗi dòng chặn một rủi ro

```
def after_task(playbook, task, trace, verdict):
  log_usage(playbook, trace)                        # was it activated?
  if verdict.source ≠ "tests":                  # no self-graded lessons
    return playbook
  lesson = cheap_model.reflect(trace, verdict)       # small model ok
  if not lesson.is_general(): return playbook
  cand = playbook.with_item(lesson,
      provenance=task.id, expires="90d")
  if val(cand) > val(playbook) + MARGIN:             # validation set
    return cand.commit()                          # versioned, revertible
  return playbook
# weekly: score the CURRENT playbook once on a
# separate TEST set never used for accept/reject
```

Mỗi dòng ứng với một bài học

chỉ học từ **kiểm thử**, không từ lời tự chấm
mô hình rẻ đủ để **viết** bài học
sửa **từng mục**, không viết lại cả playbook
ghi **có được dùng không**

Hai tập khác nhau — tập kiểm định dùng để nhận hay loại; tập kiểm tra (test set) riêng dùng để báo cáo.

Phục vụ luận điểm 3

Vòng này không được sửa quyền, sandbox hay mục tiêu; những thay đổi đó đi qua người duyệt.

Câu hỏi của chương: **Làm sao biết một hệ đa agent đúng, và giữ nó an toàn?**

Chấm đáp án cuối là chưa đủ: hệ đa agent cần đánh giá cả quỹ đạo thực thi (trajectory), truy vết từng bước và giới hạn quyền theo từng agent, vì rủi ro bảo mật cộng dồn qua ranh giới giữa các agent.

1. **Đánh giá bốn tầng.** Kết quả cuối đúng vẫn có thể do may: chỉ tầng quỹ đạo phát hiện được số liệu lấy từ nguồn không rõ.

2. **Truy vết từng bước.** Ba agent thực thi chạy song song xong sau khoảng 11,2 s thay vì khoảng 22 s; agent gọi tìm kiếm 9 lần cần xem trước.

3. **Giới hạn quyền theo agent.** Lethal trifecta (ba điều kiện cùng lúc gây rò rỉ dữ liệu) có thể chia cho ba agent; người duyệt hành động không đảo ngược được.

Sau chương này: thiết kế bộ đánh giá bốn tầng, đọc một vết thực thi để tìm agent và bước sai, và chia quyền theo từng agent.

Đáp án đúng vẫn có thể do may: cần chấm cả quỹ đạo

1. **Kết quả: đúng hoặc sai, kiểm tra tự động được**

2. **Quỹ đạo**: bước nào sai, lần chuyển giao nào làm rơi thông tin

3. **LLM-as-judge** (dùng mô hình chấm đầu ra) theo thang chấm: độ phủ, mạch lạc, giữ ý kiến bất đồng

4. **Người** chấm mẫu để hiệu chỉnh tầng 3

Ví dụ: hệ nghiên cứu 3 agent

Kết quả đúng nhưng agent thực thi thứ hai lấy số từ một blog không nguồn, agent điều phối không phát hiện. Chỉ tầng 2 bắt được lỗi này; lần sau kết quả có thể sai.

Đối chứng bắt buộc

So với **một agent tốt nhất, cùng công cụ, cùng tổng token**. Báo cáo tỷ lệ thành công, **chi phí trên mỗi câu trả lời được chấp nhận**, độ trễ, số lời gọi công cụ và phương sai qua ≥ 3 lần chạy.

Phục vụ luận điểm 1

LLM-as-judge ưu tiên câu trả lời dài và tự tin; cần đối chiếu với vài chục mẫu do người chấm (Ngày 24) trước khi tin tầng 3.

Nguồn: MultiAgentBench, arXiv:2503.01935; quy trình LLM-as-judge của MAST, arXiv:2503.13657.

Chạy song song: 11,2 s; tuần tự: khoảng 22 s

```
invoke_agent lead                                  12.4s 43k tok
  chat plan                                          1.1s 3k
  invoke_agent worker-1 (filings)                    6.0s 12k
    execute_tool web_search x4
    execute_tool fetch x3
  invoke_agent worker-2 (news)                       7.9s 15k
    execute_tool web_search x9                        # loop?
  invoke_agent worker-3 (reports)                    5.2s 9k
  chat verify                                        1.4s 2k
  chat synthesize                                    0.8s 2k
```

Tên span theo quy ước OpenTelemetry GenAI (`invoke_agent`, `execute_tool`, `chat`).

Đọc vết thực thi này

worker-2 gọi tìm kiếm 9 lần: ứng viên của kiểu lỗi lặp lại bước, phổ biến nhất theo MAST; cũng tốn nhiều token nhất (15k). $1{,}1 + \max(6{,}0;\ 7{,}9;\ 5{,}2) + 1{,}4 + 0{,}8 \approx 11{,}2$ s (12,4 s kể cả chi phí phụ), so với khoảng 22 s nếu chạy tuần tự.

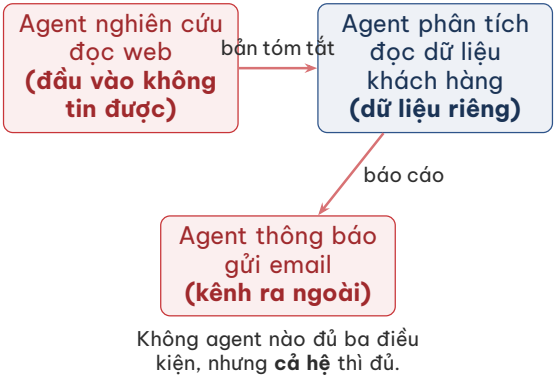
Lưu ý: Quy ước `gen_ai.*` của OpenTelemetry vẫn ở trạng thái **Development** (10/2026): ghim phiên bản, chuẩn bị đổi tên thuộc tính.

Phục vụ luận điểm 2

Vết thực thi chuyển câu hỏi "hệ có sai không" thành "agent nào sai, ở bước nào".

Nguồn: github.com/open-telemetry/semantic-conventions-genai (mọi span: Status Development, truy cập 05/10/2026).

Ba chân lethal trifecta có thể nằm ở ba agent



Không agent nào đủ ba điều kiện, nhưng **cả hệ** thì đủ.

Lethal trifecta

đầu vào không tin được + **truy cập dữ liệu riêng** + **kênh gửi ra ngoài** = đường rò rỉ qua tấn công chèn lệnh (prompt injection). Trong hệ đa agent, ba chân có thể nằm ở ba agent, nối bằng bản tóm tắt.

Phòng vệ

Tin nhắn giữa các agent là **dữ liệu có nguồn gốc**, không phải lệnh. Tên vai trò không phải thẩm quyền. Hành động có hậu quả cần **uỷ quyền riêng**, nằm ngoài mô hình.

Bằng chứng 09/2026

Trong chuỗi điều phối, nghiên cứu, kiểm tra, thực thi, kiểm soát uỷ quyền ở tầng cấu trúc vẫn chặn thực thi nguy hiểm **kể cả khi agent kiểm tra đã bị chiếm quyền**.

Nguồn: Willison, *The lethal trifecta* (2025); Safin et al., arXiv:2609.17648 (15/09/2026); chi tiết: Ngày 24.

Người chỉ duyệt hành động không đảo ngược được

Hành động	Ai quyết
đọc, tìm kiếm, tóm tắt	agent, tự động
sửa tệp trong sandbox	agent; kiểm thử làm cổng
hợp nhất vào nhánh chính	cổng tự động và người lấy mẫu
gửi ra ngoài, chi tiền, xoá	**người duyệt từng lần**
đổi quyền, đổi mục tiêu	**người**, có ghi nhật ký

Người duyệt ở đâu

Có con người phê duyệt (human-in-the-loop) với hàng trăm agent: duyệt từng lời gọi công cụ là không khả thi. Hướng 2026: người duyệt **kế hoạch** và **hành động không đảo ngược được**; sandbox và kiểm thử lo phần còn lại.
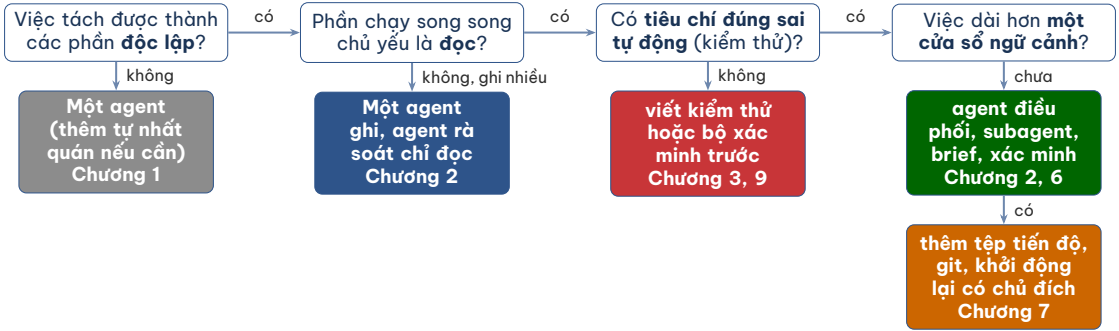
Lưu ý: Đội agent có thể đạt kết quả tốt hơn nhưng kém căn chỉnh (alignment) hơn một agent đơn: ý kiến phản đối vì lý do đạo đức có thể bị cả nhóm bỏ qua. Cần đo cách hệ **xử lý ý kiến phản đối**.

Phục vụ luận điểm 3

Quyền chia theo từng agent; người chỉ xuất hiện ở điểm không quay lại được.

Nguồn: Anthropic, *Trustworthy agents in practice* (09/04/2026); arXiv:2604.10290; chi tiết: Ngày 27.

Bốn câu hỏi dẫn tới thiết kế; mặc định một agent



Mọi nhánh kết thúc ở harness

Giới hạn bước và token, quyền, sandbox, vết thực thi (Chương 4, 9).

Và một phép đo

So với một agent tốt nhất cùng ngân sách; nếu không thắng rõ, giữ hệ đơn giản hơn (Chương 1, 2).

LIVE DEMO

1. Câu hỏi: “So sánh 5 nhà cung cấp kho vector về giá, giới hạn và giấy phép, theo nguồn sơ cấp”: việc **đọc rộng, tách được**
2. Chạy **một agent** và **một agent điều phối cùng 5 subagent** với **cùng trần token**; đặt hai vết thực thi cạnh nhau
3. Đọc vết: subagent nào lặp lại tìm kiếm; brief nào thiếu phạm vi (“SCOPE”) khiến hai nhánh làm trùng
4. Đổi sang việc **tuần tự** (kế hoạch di chuyển hệ thống gồm 6 bước) và chạy lại; giả thuyết: lợi thế của nhiều agent giảm; ghi kết quả kể cả khi trái giả thuyết
5. Đối chiếu kết quả với luận điểm của Chương 1 và Chương 2 trước khi kết luận

LAB #20

Mục tiêu: **Agent tự tiến hoá**: dựng harness bằng **Deep Agents**, rồi so ba điều kiện — `baseline`, `subagents`, `skills-auto` (skill do curator tự viết) — trên 6 tác vụ có chấm tự động.

Deliverable: Repo nhóm (1 hoặc 2–3 sinh viên): 4 file đã cài trong `src/lab/`, `skills/auto/`, `results/` (run.json và trace.md), `report/REPORT.md` cùng `table.md`.

Thời gian: Buổi thực hành 2 giờ: cài harness, chạy, nháp báo cáo mục 1–7; hoàn thiện mục 8–10 sau buổi.

Ba điều kiện × sáu tác vụ, chấm tự động

Điều kiện	Agent
`baseline`	Deep Agents mặc định, không skill: mốc so sánh
`subagents`	thêm 2–3 subagent do nhóm tự định nghĩa (Chương 2, 6)
`skills-auto`	thêm skill do curator viết từ phản hồi (Chương 8)

Nhóm	Việc (1 tác vụ học, 1 tác vụ đánh giá)
`code`	sửa gói Python có kiểm thử lỗi, lỗi gốc ở hàm dùng chung
`data`	trả lời từ CSV hoặc JSON bẩn: trùng, thiếu, ngày, múi giờ
`logs`	phân tích nhật ký: lỗi nhiều dòng, dòng lặp, mức nhật ký

Điểm = kiểm tra đạt / tổng kiểm tra

Mỗi tác vụ có `check.py` trả danh sách kiểm tra đạt hoặc không đạt, tính điểm từng phần. Tác vụ đánh giá cùng loại việc, khác dữ liệu, thêm một quy ước mới.

Quy ước chỉ học được từ phản hồi

Kiểm tra tên `rule_` đo quy ước của tổ chức giả định "Acme" mà đề không nêu. Ở tác vụ học, kiểm tra thất bại trả `detail` nêu quy tắc, không chứa đáp án; ở tác vụ đánh giá, `detail` rỗng.

Lưu ý: Với mô hình mạnh, lỗi thường dồn vào quy ước tổ chức; đó cũng là một phát hiện, báo cáo bằng số.

Mốc đóng băng chặn việc chọn skill theo điểm



`verify_freeze.py` phải báo đạt

Có lần lưu `hypotheses` (H1–H3) **trước** tag; skill không đổi từ tag; mọi lần chạy `skills-auto` dùng skill đã đóng băng.

H1–H3 neo vào hai nghiên cứu

SkillsBench v1: skill người viết +16,2 điểm phần trăm, skill mô hình tự sinh trung bình không lợi; v4: skill tuyển chọn 33,9% → 50,5%. SkillEvolBench: lợi ích không bền sau đóng băng.

Ý của frame

Tác vụ học là tập chọn, tác vụ đánh giá là held-out để báo cáo (Chương 8).

Nguồn: README §2.1, GUIDE Phần 4; arXiv:2602.12670 (v1 13/02/2026, v4 14/06/2026); 2605.24117 (22/05/2026).

Bốn file cần cài, mỗi file gắn một chương

Tệp cần hoàn thành	Việc	Kiểm thử offline	Chương
`subagents.py`	2–3 subagent (khảo sát, cài đặt, rà soát); `description` nói *khi nào* gọi	`test_02`	2, 6: brief, cách ly ngữ cảnh
`agent.py`	`make_backend` (sandbox, `PATH`, không kế thừa biến môi trường); `build_agent`	`test_02` (9 test)	4: harness
`runner.py`	`run_task`: sandbox mới, đếm token và lời gọi công cụ, `skills_read`; băm `skills/`	`test_03` (6 test)	9: vết thực thi, đo lường
`curator.py`	`curate_skills`: chỉ đọc lần chạy **học**, gọi mô hình một lần, ghi skill hợp lệ	`test_04` (2 test)	8: tự tiến hoá

Lưu ý: `inherit_env=True` cho phép shell của agent đọc **khoá API**: đây là chân "dữ liệu riêng" của lethal trifecta (Chương 9). Shell không có `/workspace`: dùng đường dẫn tương đối.

`validate_skill` (có sẵn) chặn rò rỉ

Tên ngoài `[a-z0-9-]` (ví dụ `../evil`); thân dài hơn 80 dòng; thiếu `description`; mọi định danh chỉ có ở tác vụ đánh giá. Không sửa tay `skills/auto/`.

Nguồn: Lab repo: guides/pseudocode/01–05; tests/; ghim `deepagents==0.7.21`.

Điểm lab dựa trên phân loại lỗi có bằng chứng

Phân loại lỗi (mỗi dòng cần bằng chứng)	
A	bỏ qua đặc tả (README, docstring)
B	không kiểm chứng (không chạy lại kiểm thử)
C	vá triệu chứng thay vì nguyên nhân gốc
D	bỏ sót dữ liệu bẩn, định dạng, múi giờ
E	vi phạm quy ước tổ chức (`rule_`, `RULE:`)
F	báo hoàn thành sai sự thật (MAST, Chương 3)
G	khác

Thang điểm	điểm
Harness (pytest tự động)	30
Tác vụ học, baseline, phân loại lỗi	14
Subagent: thiết kế, nhận xét từ vết thực thi	10
Curator và đánh giá skill	16
Bảng so sánh, `freeze` đạt	10
Báo cáo: giả thuyết, phân tích, hạn chế	20
Thưởng: 1 thử thách mở rộng	+5

Trừ 10 điểm cho mỗi lỗi

Lộ khoá API; sửa `tests/` hoặc `tasks/`; báo kết quả như thể skill không đổi sau tag. `subagent_calls = 0` là kết quả hợp lệ nếu giải thích được.

Lưu ý: Repo chưa được chạy thử. GUIDE trỏ tới README §2.3 (quy tắc xem dữ liệu) và §7, hai mục không tồn tại; rubric phạt việc mở tệp đánh giá trước mốc đóng băng nhưng quy tắc đó chưa được viết ra.

Nguồn: Lab repo: GUIDE Phần 2.2–2.3, RUBRIC.

QUIZ Tuần 4 và MILESTONE 1 nộp sau một tuần

QUIZ Tuần 4 (20 phút)

10 câu: 7 trắc nghiệm và 3 trả lời ngắn

Phạm vi: Reflexion, bộ nhớ, tạo sinh tăng cường truy xuất (RAG) cho môi trường vận hành, GraphRAG, hệ đa agent và harness

Kiểm tra hiểu, không kiểm tra nhớ cú pháp. Câu mẫu: frame "Kiểm tra nhanh" ở Chương 3.

MILESTONE 1: hồ sơ Ngày 16–20

Nộp hồ sơ gồm năm lab:

- Lab 16: agent Reflexion
- Lab 17: hệ thống bộ nhớ
- Lab 18: RAG cho môi trường vận hành
- Lab 19: GraphRAG
- Lab 20: agent tự tiến hoá (Deep Agents)

Hạn: 1 tuần sau Ngày 20

Tổng kết: mỗi chương dạy một luận điểm

Chương 1:	chỉ tách nhiều agent khi việc chia được thành phần độc lập hoặc vượt một cửa sổ ngữ cảnh.
Chương 2:	chọn mẫu theo phụ thuộc giữa các tác vụ; mẫu phức tạp phải thắng đối chứng đơn giản trên cùng số câu trả lời.
Chương 3:	ba nhóm lỗi (thiết kế, phối hợp, xác minh) đều can thiệp được ở tầng hệ thống; chẩn đoán theo MAST trước khi đổi mô hình.
Chương 4:	agent là mô hình cộng harness; cùng mô hình, đổi harness thì kết quả đo được đổi.
Chương 5:	DeepSeek Harness dựng hoàn toàn từ plugin nên thay được mọi thành phần; hiện là thiết kế tham chiếu để học.
Chương 6:	giữa các agent, ngữ cảnh đi qua brief và kết quả trả về, dữ liệu lớn đi qua tệp; thiết kế ngữ cảnh là chọn chuyển, cắt, giữ.
Chương 7:	agent chạy hàng giờ khi harness giữ trạng thái ngoài mô hình và dừng bằng cổng kiểm tra.
Chương 8:	agent tự tiến hoá thường sửa thành phần ngoài mô hình (skill, bộ nhớ, mã); chỉ giữ thay đổi vượt kiểm tra trên dữ liệu agent không đọc được, báo cáo trên held-out riêng, vì rủi ro chính là reward hacking.
Chương 9:	đánh giá cả quỹ đạo thực thi, truy vết từng bước và giới hạn quyền theo từng agent.
Chương 10:	dùng cây quyết định để chọn thiết kế, rồi đo bằng đối chứng cùng ngân sách như trong Lab 20.

Bài tiếp theo

Ngày 21: tinh chỉnh (fine-tuning) mô hình ngôn ngữ lớn (LLM) bằng LoRA/QLoRA

"Ngày 20 cải tiến harness xung quanh mô hình. Ngày 21 xét khi nào phải cải tiến chính trọng số, và khi nào chỉnh prompt, ngữ cảnh hoặc skill là đủ."

Bài tập về nhà

- Hoàn thành Lab 20 (báo cáo mục 8–10), nộp Milestone 1
- Đọc MAST (arXiv:2503.13657) mục 3–4 và *Effective harnesses for long-running agents*
- Tuỳ chọn: chạy harness tối giản (mini-SWE-agent hoặc Pi) trên một kho mã nhỏ

Tài liệu tham khảo

1. Kim et al. *Towards a Science of Scaling Agent Systems*. arXiv:2512.08296 v3. Cemri et al. *Why Do Multi-Agent LLM Systems Fail?* NeurIPS 2025 D&B. arXiv:2503.13657.
2. Anthropic. *How we built our multi-agent research system* (13/06/2025); *Building effective agents* (19/12/2024); *Effective context engineering for AI agents* (29/09/2025); *Writing effective tools for agents* (11/09/2025).
3. Cognition. *Don't Build Multi-Agents* (06/2025); *Multi-Agents: What's Actually Working* (22/04/2026).
4. Lewis. *Same Model, Different Harness*. arXiv:2608.26218. *Harness-Bench*. arXiv:2605.27922. *Harness Updating Is Not Harness Benefit*. arXiv:2605.30621.
5. OpenAI. *Harness engineering: leveraging Codex in an agent-first world* (11/02/2026); *Unlocking the Codex harness* (04/02/2026). Manus. *Context Engineering for AI Agents: Lessons from Building Manus* (2025).
6. DeepSeek-AI. *DeepSeek Harness*, bản xem trước dành cho nhà phát triển: github.com/deepseek-ai/deepseek-harness (truy cập 05/10/2026). *DeepSeek-V3.2*. arXiv:2512.02556.
7. Kho mã: SWE-agent/mini-swe-agent; earendil-works/pi; openai/codex (truy cập 05/10/2026).
8. Kwa et al. (METR). *Measuring AI Ability to Complete Long Tasks*. arXiv:2503.14499; metr.org/time-horizons.
9. Anthropic. *Effective harnesses for long-running agents* (26/11/2025); *Building a C compiler with a team of parallel Claudes* (05/02/2026). Cursor. *Scaling long-running autonomous coding* (14/01/2026); *Towards self-driving codebases* (05/02/2026).
10. *AI Organizations are More Effective but Less Aligned than Individual Agents*. arXiv:2604.10290. Su et al. *ToolOrchestra*. arXiv:2511.21689.
11. Gao et al. arXiv:2507.21046 (khảo sát). Zhang et al. *ACE*. arXiv:2510.04618. Agrawal et al. *GEPA*. arXiv:2507.19457.
12. *Darwin Gödel Machine*. arXiv:2505.22954. *SICA*. arXiv:2504.15228. *Agent0*. arXiv:2511.16043. Srikanth et al. (AIDE^2). arXiv:2609.26457. *SkillsBench*. arXiv:2602.12670. *SkillEvolBench*. arXiv:2605.24117. Safin et al. arXiv:2609.17648.
13. Huang et al. *LLMs Cannot Self-Correct Reasoning Yet* (ICLR 2024); Smit et al. (ICML 2024).

Hỏi & Đáp

Với bài toán ở đơn vị công tác: dùng một agent hay nhiều agent, và harness còn thiếu mục nào trong danh sách kiểm tra?

VINUNIVERSITY

Cảm ơn!

AICB-P2T3 · Ngày 20 · Hệ đa agent, harness và agent tự tiến hoá

github.com/vinuni-aicb

Liên hệ: instructor@vinuni.edu.vn